Informe de Proyecto: Alke Wallet

Autor: Ignacio Trujillo

Módulo: Desarrollo Web con Django – Integración con Bases de Datos

Herramientas utilizadas: Visual Studio Code, Django

Fecha: 31 de marzo de 2026

1. Introducción

Este proyecto consiste en el desarrollo de una aplicación web en Django denominada **Alke Wallet**, cuyo objetivo es gestionar información persistente mediante el uso de modelos ORM, relaciones entre entidades y operaciones CRUD.

2. Objetivo del proyecto

El objetivo de este proyecto es el desarrollo una aplicación web en Django que permita gestionar información persistente mediante base de datos SQLite, utilizando modelos ORM, relaciones entre entidades, migraciones y operaciones CRUD completas integradas con vistas y templates.

3. Tecnologías utilizadas

- Python
- Django
- Visual Studio Code
- HTML
- CSS
- Bootstrap
- SQLite

4. Estructura de trabajo

Etapa 1 - Creación y configuración inicial del proyecto

- Creación de carpeta y repositorio del proyecto
- Creación y activación del entorno virtual
- Instalación de Django y verificación de versión
- Creación del proyecto “alke_wallet”
- Creación de la aplicación “wallet”

Etapa 2 - Configuración inicial de la aplicación

- Registro de la app en “INSTALLED_APPS”
- Verificación de la base de datos SQLite
- Organización de carpetas para templates y archivos estáticos
- Prueba inicial del servidor de desarrollo

Etapa 3 - Modelado de datos (ORM)

- Definición de modelos principales: Usuario, Moneda y Transacción

- Establecimiento de relaciones mediante claves foráneas (ForeignKey)
- Implementación de métodos __str__ para representación de datos

Etapa 4 - Migraciones

- Generación de archivos de migración (makemigrations)
- Aplicación de migraciones a la base de datos (migrate)
- Sincronización y verificación de la estructura en SQLite

Etapa 5 - Administración (Django Admin)

- Registro de modelos en admin.py
- Creación de superusuario para acceso al panel
- Carga y gestión de datos de prueba desde el panel administrativo

Etapa 6 - Implementación de CRUD de Usuarios

- Creación de vistas basadas en clases (ListView, CreateView, UpdateView, DeleteView)
- Configuración de rutas específicas en urls.py
- Desarrollo de templates para el ciclo CRUD de usuarios

Etapa 7 - Implementación de CRUD de Transacciones

- Integración de vistas para gestión de transacciones relacionadas
- Actualización de rutas y navegación del sistema
- Validación de relaciones (Usuario y Moneda) en formularios dinámicos

Etapa 8 - Interfaz Visual y Estilos

- Incorporación del framework Bootstrap
- Creación de hoja de estilos personalizada (style.css)
- Mejoras en la experiencia de usuario (cards, botones y fondos)

Etapa 9 - Integración Final y Verificación General

- Consolidación de la navegación mediante el template base
- Validación del flujo completo de la aplicación y persistencia de datos
- Pruebas de integridad en las relaciones entre modelos

5. Desarrollo del proyecto

5.1 Creación y configuración inicial del proyecto

En esta etapa se preparó el entorno de desarrollo para la construcción del proyecto **Alke Wallet**, utilizando el framework Django.

Para ello se creó la carpeta inicial y se inició el repositorio del proyecto.

Ya definido esto, se creó un entorno virtual mediante Python utilizando el comando **python -m venv venv**, con el fin de aislar las dependencias del proyecto y evitar conflictos con otras instalaciones del sistema. Posteriormente, se activó el entorno virtual, lo que permitió trabajar en un entorno controlado.

Luego, se instaló Django mediante el gestor de paquetes pip, utilizando el comando **pip install django**. Una vez instalado, se verificó correctamente la versión instalada mediante el comando **django-admin –version**, confirmando que el framework estaba disponible para su uso.

A continuación, se procedió a la creación del proyecto base utilizando el comando **django-admin startproject alke_wallet .**, lo que generó la estructura inicial del proyecto, incluyendo archivos fundamentales como **manage.py**, **settings.py** y **urls.py**.

Finalmente, se ejecutó el servidor de desarrollo mediante el comando **python manage.py runserver**, verificando en el navegador que la aplicación funcionara correctamente al mostrar la página inicial de Django. Esto confirmó que la configuración inicial del proyecto se realizó de manera exitosa.

1. Creación de carpeta y repositorio del proyecto

Repositorio del proyecto

https://github.com/ignaciotrujilloleiva/Proyecto-Modulo-7-Desarrollo-web-Django-Integraci-n-Base-de-datos.git

Carpeta del proyecto

Proyecto-Modulo-7-Desarrollo-web-Django-Integración-Base-de-datos

2. Creación y activación del entorno virtual

Comandos utilizados:

- **python -m venv venv**
- **venv\Scripts\activate**

3. Instalación de Django y verificación de la versión

Comandos utilizados:

- pip install django
- **django-admin –version**

4. Creación del proyecto Alke Wallet

Comandos utilizados:

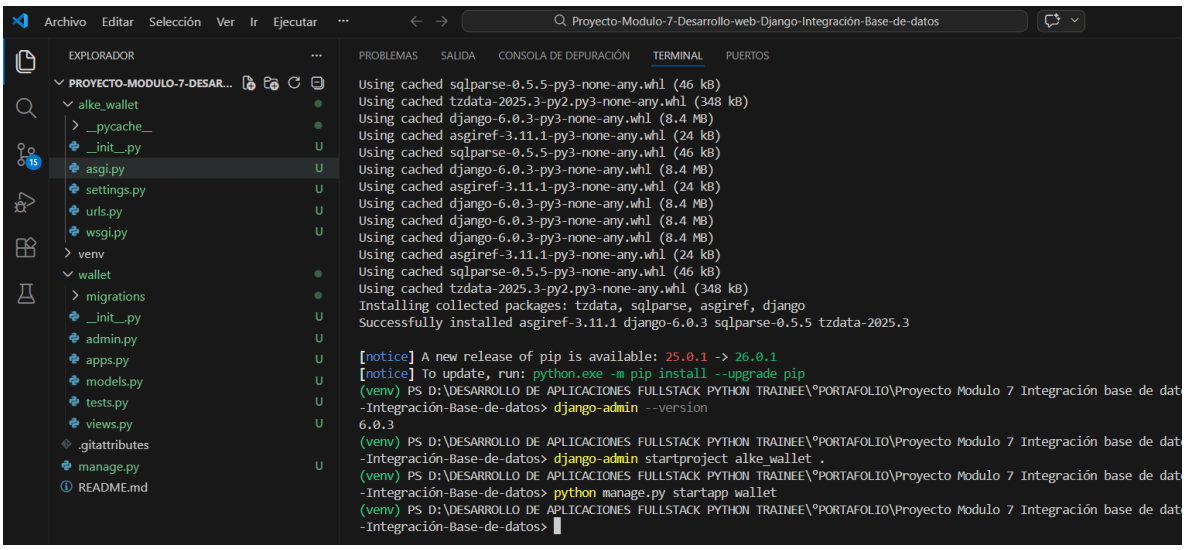
- django-admin startproject alke_wallet .

5. Creación de la app wallet

Comandos utilizados:

- Python manage.py startapp wallet
-

Inicio del proyecto y aplicación ya creada



5.2 Configuración inicial de la aplicación

En esta etapa se configuró la aplicación principal del proyecto llamada **wallet**, la cual contendrá la lógica central del sistema **Alke Wallet.**

Esta aplicación será la encargada de gestionar los modelos, vistas, rutas y demás componentes necesarios para el funcionamiento del sistema.

La creación de la aplicación generó automáticamente una estructura base compuesta por archivos como **models.py**, **views.py**, **admin.py** y **apps.py**, los cuales cumplen funciones específicas dentro de la arquitectura del framework Django.

Posteriormente, se procedió a registrar la aplicación en el archivo **settings.py**, agregándola a la lista **INSTALLED_APPS**. Este paso es fundamental, ya que permite que Django reconozca la aplicación como parte del proyecto y habilite su funcionamiento.

Además, se organizó la estructura interna de la aplicación creando las carpetas necesarias para el manejo de templates y archivos estáticos dentro de la misma app.

Finalmente, se ejecutó el servidor de desarrollo para verificar que la aplicación se integrara correctamente al proyecto sin generar errores.

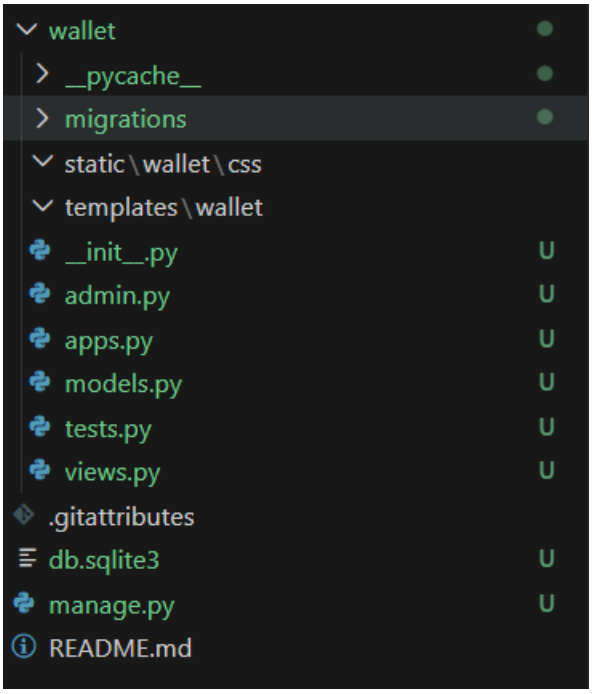
1. Registro de la aplicación en settings.py

```
settings.py U ×
alke_wallet > settings.py > ...
30
31  # Application definition
32
33  INSTALLED_APPS = [
34      'django.contrib.admin',
35      'django.contrib.auth',
36      'django.contrib.contenttypes',
37      'django.contrib.sessions',
38      'django.contrib.messages',
39      'django.contrib.staticfiles',
40      'wallet',
41  ]
```

2. Verificación de base de datos SQLite

```
settings.py U ×
alke_wallet > settings.py > ...
72
73  # Database
74  # https://docs.djangoproject.com/en/6.0/ref/settings/#databases
75
76  DATABASES = {
77      'default': {
78          'ENGINE': 'django.db.backends.sqlite3',
79          'NAME': BASE_DIR / 'db.sqlite3',
80      }
81  }
82
```

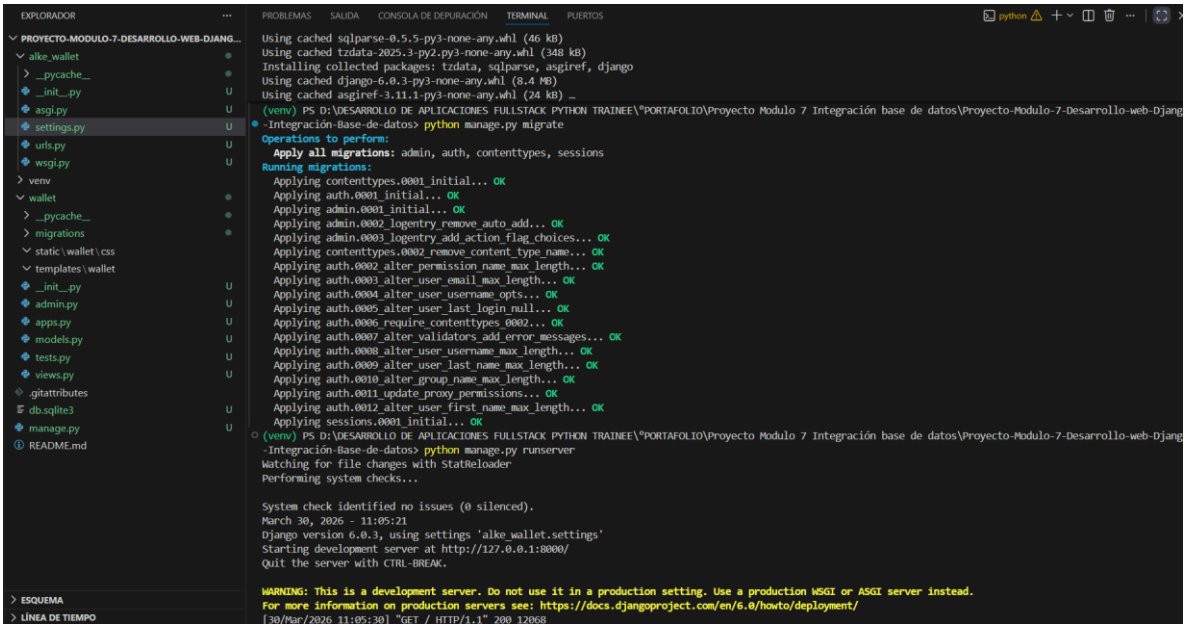
3. Creación de carpetas template y static

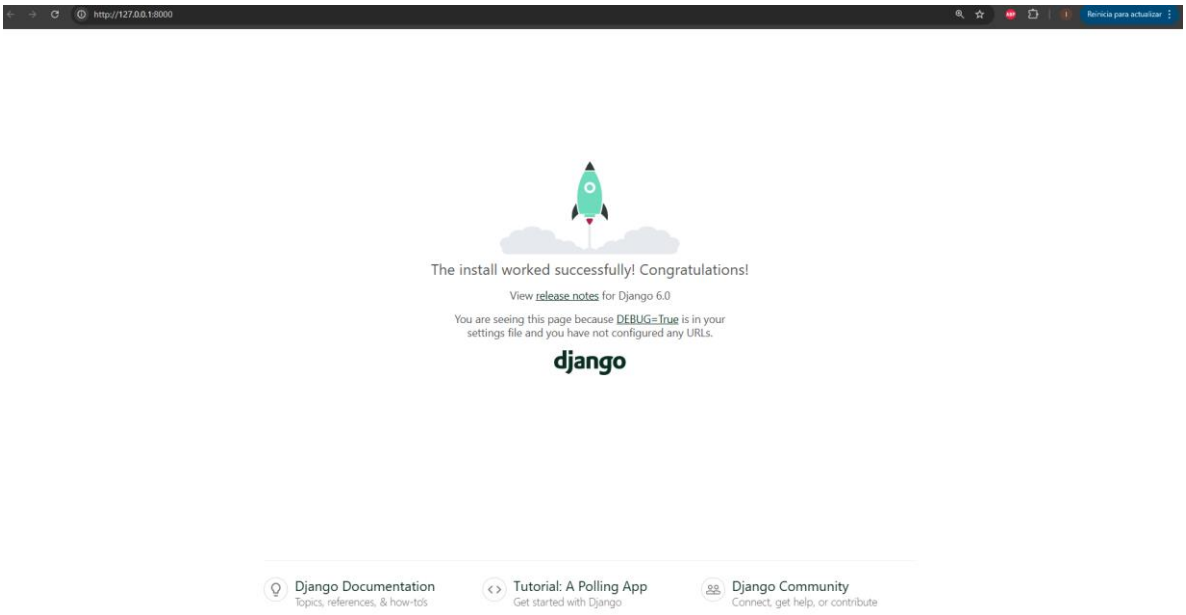


4. Verificación de funcionamiento

Comandos utilizados:

- python manage.py migrate
- python manage.py runserver





5.3 Modelado de datos (ORM)

En esta etapa se realizó el diseño y definición de los modelos de datos del proyecto **Alke Wallet**, utilizando el sistema ORM (Object Relational Mapping) de Django. Este permite representar las tablas de la base de datos como clases en Python, facilitando la manipulación de datos sin necesidad de escribir consultas SQL directamente.

Se definieron tres modelos principales: **Usuario**, **Moneda** y **Transaccion**, los cuales representan las entidades fundamentales del sistema. Cada modelo fue implementado en el archivo **"models.py"** de la aplicación **"wallet"**, definiendo sus atributos mediante campos proporcionados por Django, tales como CharField, EmailField, DecimalField y DateTimeField.

Además, se establecieron relaciones entre los modelos mediante el uso de claves foráneas (**ForeignKey**), permitiendo representar las asociaciones entre entidades. En este caso, cada transacción está asociada a un usuario y a una moneda, lo que permite mantener la integridad de los datos y modelar correctamente el comportamiento del sistema.

1. Definición del modelo Usuario

Se creó el modelo **Usuario**, el cual representa a los usuarios del sistema. Este modelo incluye información básica como nombre, correo electrónico, saldo disponible y fecha de registro.

Campos definidos:

- **nombre**: almacena el nombre del usuario.
- **correo**: almacena el correo electrónico, con restricción de unicidad.
- **saldo**: almacena el saldo disponible del usuario.
- **fecha_registro**: registra automáticamente la fecha de creación.

```python
class Usuario(models.Model):
    nombre = models.CharField(max_length=100)
    correo = models.EmailField(unique=True)
    saldo = models.DecimalField(max_digits=12, decimal_places=2, default=0)
    fecha_registro = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.nombre
```

2. Definición del modelo Moneda

Se creó el modelo **Moneda**, encargado de representar los distintos tipos de moneda disponibles en el sistema.

Campos definidos:

- **nombre**: nombre de la moneda.
- **simbolo**: símbolo representativo de la moneda.

```python
class Moneda(models.Model):
    nombre = models.CharField(max_length=50)
    simbolo = models.CharField(max_length=10)

    def __str__(self):
        return f"{self.nombre} ({self.simbolo})"
```

3. Definición del modelo Transaccion

Se creó el modelo **Transaccion**, el cual representa las operaciones realizadas por los usuarios dentro del sistema.

Este modelo incluye relaciones con los modelos **Usuario** y **Moneda**, establecidas mediante claves foráneas (**ForeignKey**), lo que permite asociar cada transacción a un usuario específico y a una moneda determinada.

Campos definidos:

- **usuario**: relación con el modelo Usuario.
- **moneda**: relación con el modelo Moneda.
- **monto**: valor de la transacción.
- **tipo_transaccion**: tipo de operación (depósito, retiro o transferencia).
- **fecha**: fecha en que se realiza la transacción.

```python
models.py M
wallet > models.py > Moneda

class Transaccion(models.Model):
    TIPO_TRANSACCION = [
        ('deposito', 'Depósito'),
        ('retiro', 'Retiro'),
        ('transferencia', 'Transferencia'),
    ]

    usuario = models.ForeignKey(Usuario, on_delete=models.CASCADE, related_name='transacciones')
    moneda = models.ForeignKey(Moneda, on_delete=models.CASCADE, related_name='transacciones')
    monto = models.DecimalField(max_digits=12, decimal_places=2)
    tipo_transaccion = models.CharField(max_length=20, choices=TIPO_TRANSACCION)
    fecha = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.usuario.nombre} - {self.tipo_transaccion} - {self.monto}"
```

4. Definición de relaciones entre modelos

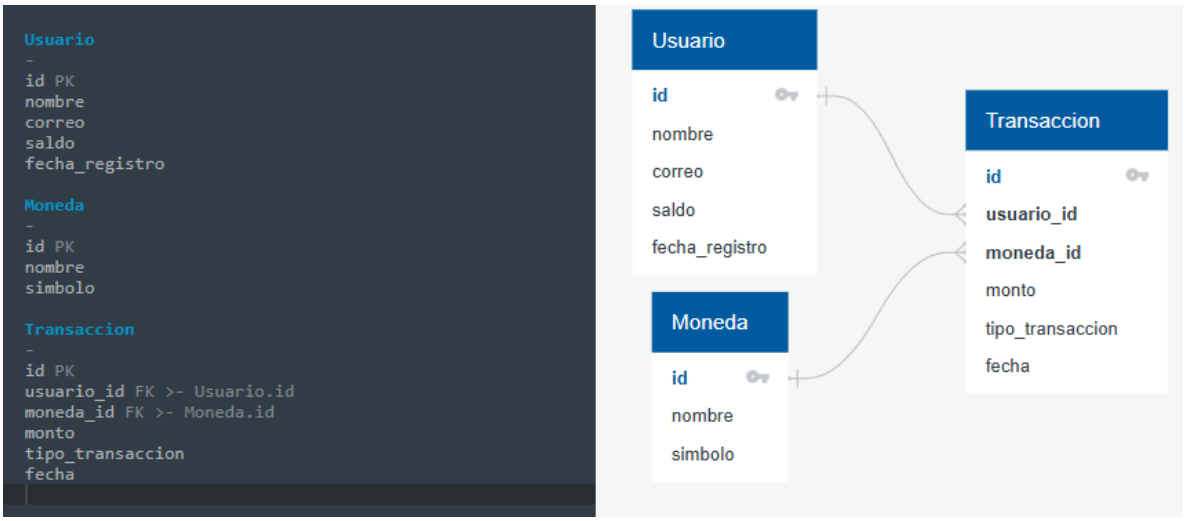
Representación de relaciones

El modelo de datos fue representado mediante un diagrama entidad-relación, en el cual se observa la relación entre las entidades principales del sistema.

En este esquema se puede identificar que:

- Un **usuario** puede estar asociado a múltiples **transacciones**, estableciendo una relación de uno a muchos (1:N).
- Una **moneda** puede estar asociada a múltiples **transacciones**, también bajo una relación de uno a muchos (1:N).

La tabla **Transaccion** actúa como entidad central, ya que contiene las claves foráneas que permiten vincular tanto al usuario como a la moneda, consolidando la lógica del sistema.



"El diagrama fue generado utilizando QuickDBD para visualizar de forma clara la estructura relacional del sistema."

5.4 Migraciones

En esta etapa se generaron y aplicaron las migraciones necesarias para traducir la estructura definida en los modelos del proyecto **Alke Wallet** a tablas dentro de la base de datos SQLite.

Una vez definidos los modelos **Usuario**, **Moneda** y **Transaccion** en el archivo **models.py**, se utilizó el comando **python manage.py makemigrations**, el cual permitió a Django detectar automáticamente los cambios realizados en la capa de modelos y generar los archivos de migración correspondientes. Estos archivos representan, en forma de instrucciones, las operaciones necesarias para crear las tablas y relaciones en la base de datos.

Posteriormente, se ejecutó el comando **python manage.py migrate**, con el objetivo de aplicar dichas migraciones en la base de datos SQLite. Este proceso crea las tablas correspondientes a los modelos definidos, además de las relaciones establecidas mediante claves foráneas.

La ejecución de las migraciones permitió sincronizar la estructura lógica del proyecto con la base de datos, asegurando que los modelos definidos en Python quedaran correctamente reflejados en el sistema de persistencia de datos. Con ello, el proyecto quedó preparado para comenzar a registrar información y avanzar hacia la implementación de las operaciones CRUD.

1. Generación de migraciones

Luego de definir los modelos del sistema en models.py, se ejecutó el comando **python manage.py makemigrations**, el cual generó automáticamente los archivos de migración en la carpeta migrations/ de la aplicación wallet.

Comandos utilizados:

- python manage.py makemigrations

```
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\°PORTAFOLIO\Proyecto Modulo 7 Integración base de datos
\Proyecto-Modulo-7-Desarrollo-web-Django-Integración-Base-de-datos> python manage.py makemigrations
Migrations for 'wallet':
  wallet\migrations\0001_initial.py
    + Create model Moneda
    + Create model Usuario
    + Create model Transaccion
```

2. Aplicación de migraciones en la base de datos

Una vez generadas las migraciones, se aplicaron mediante el comando migrate, creando así las tablas correspondientes en la base de datos SQLite.

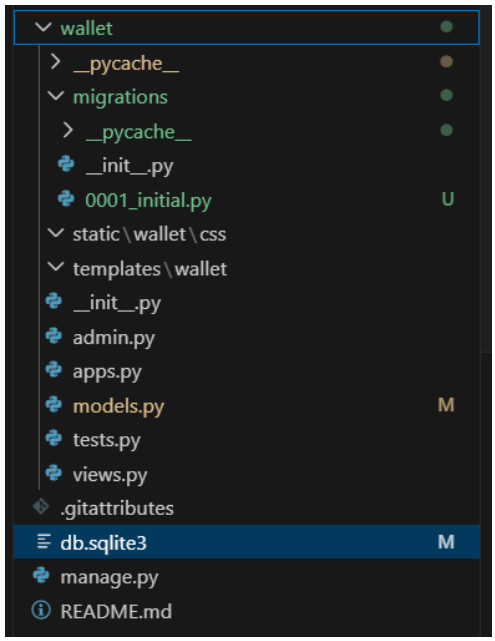
Comandos utilizados:

- python manage.py migrate

```
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\°PORTAFOLIO\Proyecto Modulo 7 Integración base de datos
\Proyecto-Modulo-7-Desarrollo-web-Django-Integración-Base-de-datos> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, wallet
Running migrations:
  Applying wallet.0001_initial... OK
```

3. Verificación de la base de datos

Como resultado de la aplicación de migraciones, se generó y actualizó el archivo **db.sqlite3**, que corresponde a la base de datos del proyecto. En esta etapa quedó persistida la estructura de datos necesaria para almacenar usuarios, monedas y transacciones.



4. Sincronización entre modelos y base de datos

Las migraciones permitieron dejar sincronizados los modelos definidos en Python con el esquema de la base de datos. Esto asegura que cualquier operación posterior realizada desde el ORM de Django se ejecute sobre una estructura de datos correctamente creada y consistente.

5.5 Administración del proyecto desde Django Admin

En esta etapa se configuró el panel de administración de Django con el objetivo de gestionar los datos del sistema **Alke Wallet**. Para ello, se registraron en el archivo **admin.py** los modelos principales del proyecto: **Usuario**, **Moneda** y **Transaccion**.

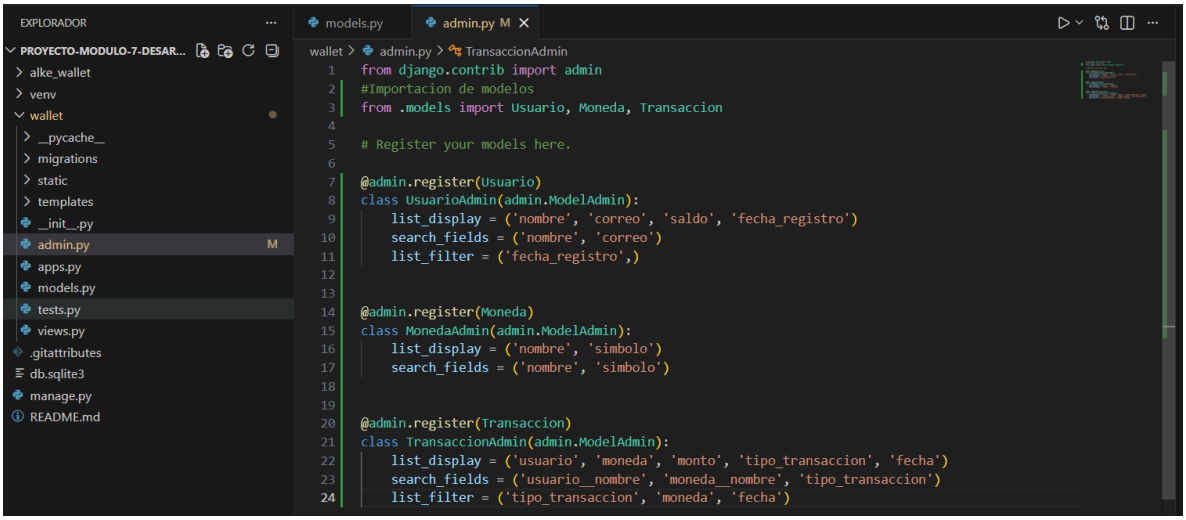
El panel administrativo de Django permite realizar operaciones CRUD sobre los modelos registrados sin necesidad de construir una interfaz personalizada desde cero. Esto permite cargar datos de ejemplo, verificar relaciones entre entidades y validar la correcta integración entre modelos y base de datos.

Posteriormente, se creó un superusuario mediante el comando **python manage.py createsuperuser**, lo que permite acceder al panel administrativo. Una vez dentro del panel, se verificó que los modelos aparecieran correctamente registrados y se procedió a ingresar datos de prueba, tales como usuarios, monedas y transacciones.

Con esta implementación, el proyecto quedó preparado para ser administrado desde el entorno interno de Django, fortaleciendo la validación del modelo de datos y facilitando las pruebas funcionales del sistema.

1. Registro de modelos en admin.py

Se registraron los modelos **Usuario**, **Moneda** y **Transaccion** en el archivo **admin.py** de la aplicación **wallet**, con el fin de habilitar su gestión desde el panel administrativo.



2. Creación de superusuario

Para acceder al panel administrativo de Django, se creó un superusuario.

Comandos utilizados:

- python manage.py createsuperuser

Para la creación del superusuario se usaron las características

- Username: admin
- Email address: admin@admin.cl
- Password: admin1234

```
(venv) PS D:\DESARROLLO DE APLICACIONES FULLSTACK PYTHON TRAINEE\°PORTAFOLIO\Proyecto Modulo 7 Integración base de datos
\Proyecto-Modulo-7-Desarrollo-web-Django-Integración-Base-de-datos> python manage.py createsuperuser
Username (leave blank to use 'ignac'): admin
Email address: admin@admin.cl
Password:
Password (again):
The password is too similar to the username.
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

3. Acceso al panel de administración

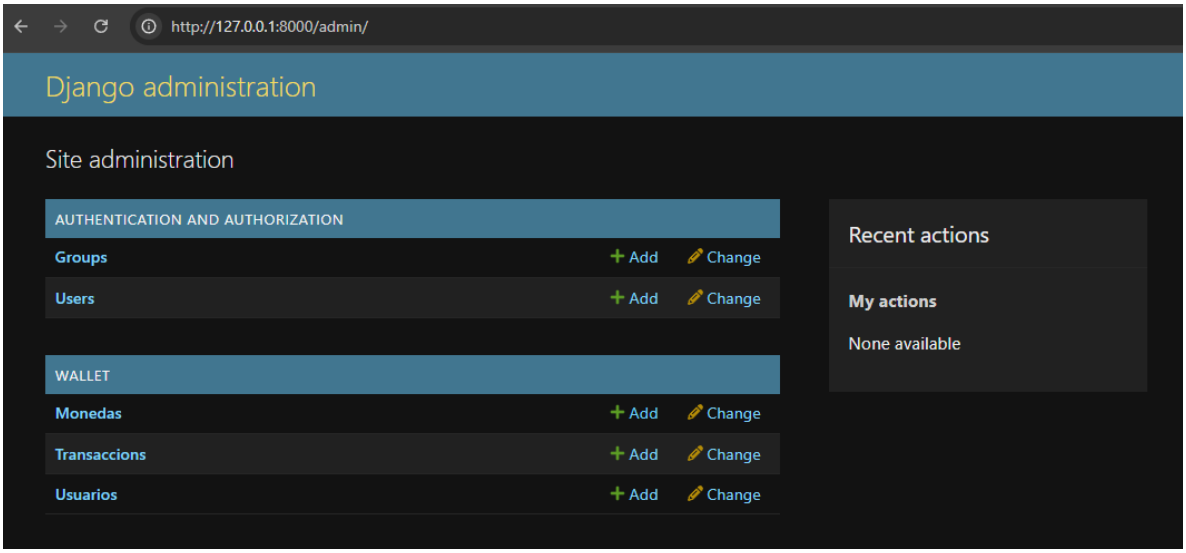
Una vez creado el superusuario, se ejecutó nuevamente el servidor de desarrollo y se accedió a la ruta /admin/ desde el navegador.

Comandos utilizados:

- python manage.py runserver

Ruta de acceso:

- http://127.0.0.1:8000/admin/



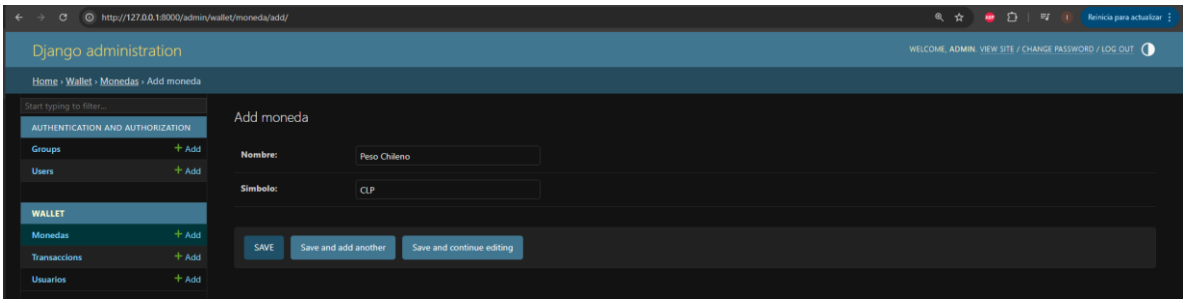
4. Carga de datos de prueba

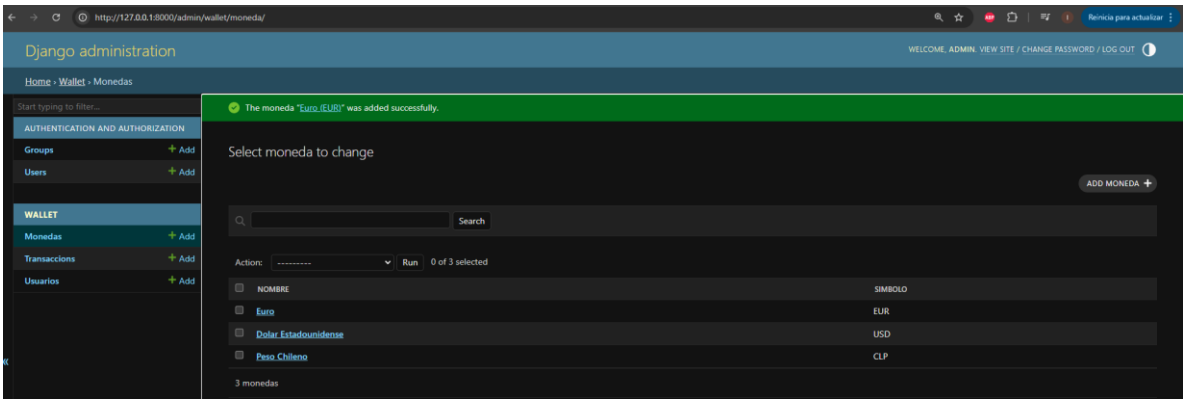
Desde el panel administrativo se ingresaron registros de prueba para validar el funcionamiento del sistema y comprobar la correcta relación entre los modelos.

Datos cargados

Monedas

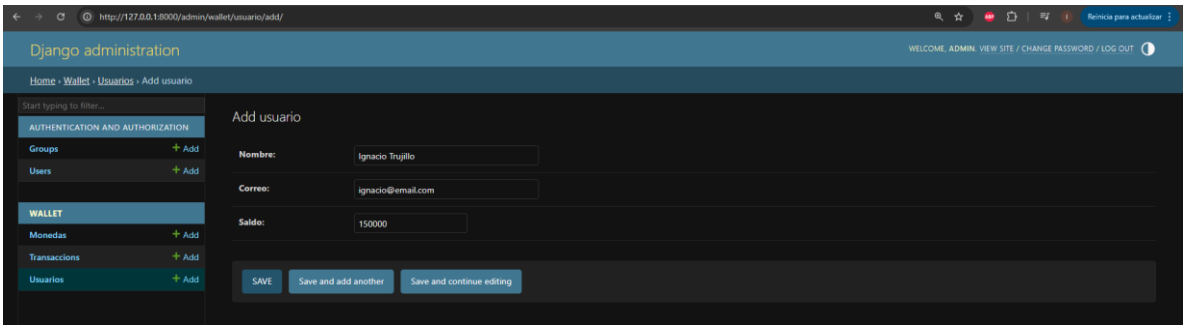
- Nombre: Peso Chileno | Símbolo: CLP
- Nombre: Dólar Estadounidense | Símbolo: USD
- Nombre: Euro | Símbolo: EUR

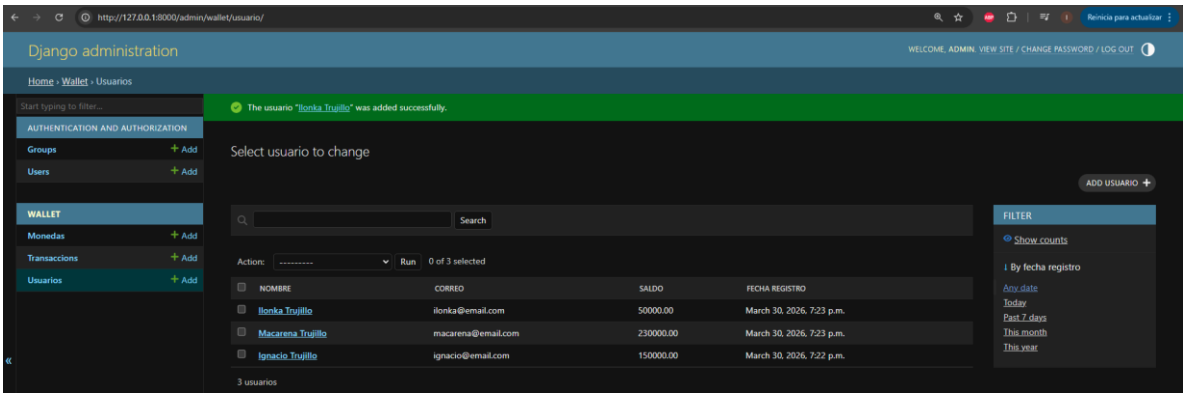




Usuarios

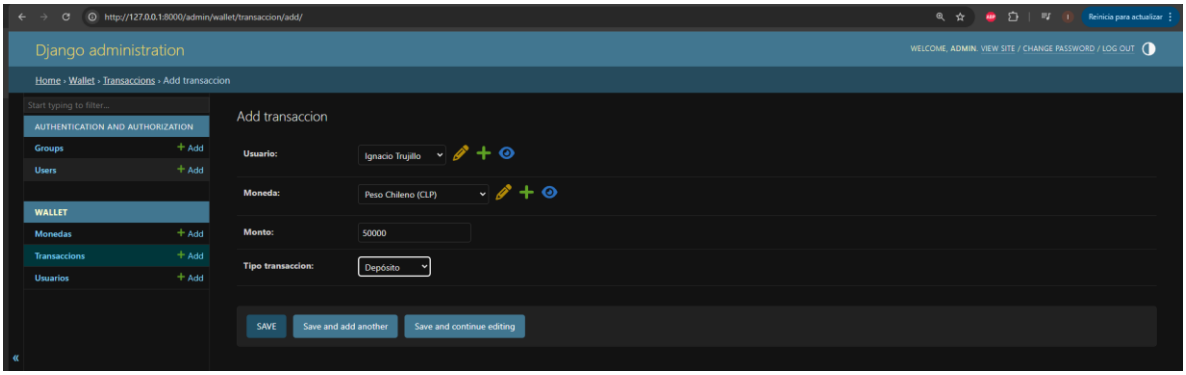
- Nombre: Ignacio Trujillo | Correo: ignacio@email.com | Saldo: 150000
- Nombre: Macarena Trujillo | Correo: macarena@email.com | Saldo: 230000
- Nombre: Ilonka Trujillo| Correo: ilonka@email.com | Saldo: 50000

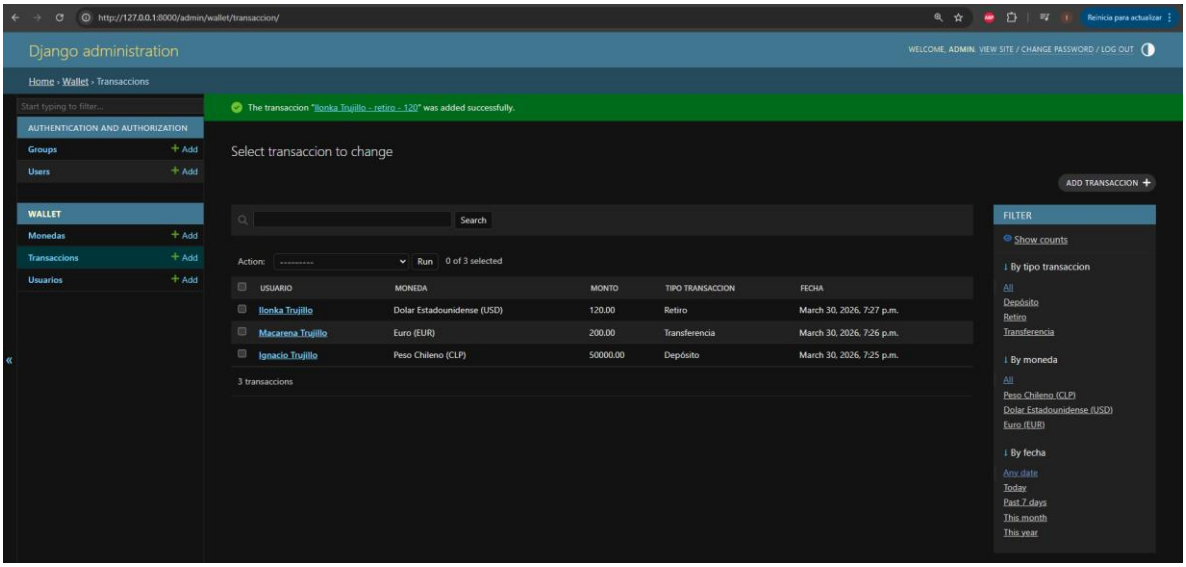




Transacciones

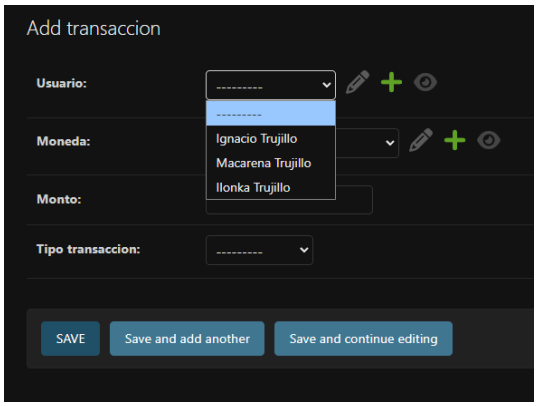
- Usuario: Ignacio Trujillo | Moneda: Peso Chileno | Monto: 50000 | Tipo: depósito
- Usuario: Macarena Trujillo | Moneda: Euro| Monto: 200 | Tipo: transferencia
- Usuario: Ilonka Trujillo | Moneda: Dólar Estadounidense | Monto: 120 | Tipo: Retiro

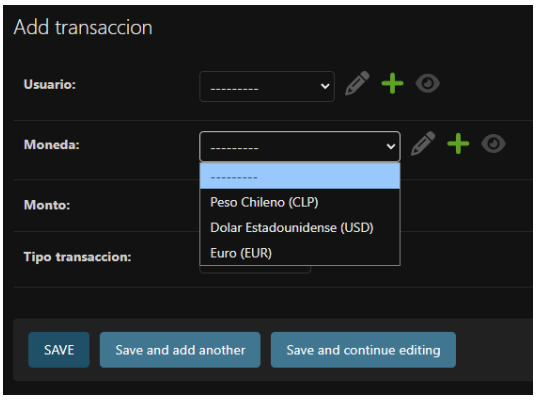




5. Validación de relaciones desde el admin

Gracias al uso de claves foráneas, el panel admin mostrará listas desplegables para seleccionar el usuario y la moneda al momento de crear una transacción. Esto permite validar visualmente la relación entre las entidades y comprobar que la estructura del modelo fue implementada correctamente.





5.6 Implementación del CRUD de usuarios

En esta etapa se implementó un sistema CRUD completo para el modelo **Usuario**, permitiendo crear, visualizar, actualizar y eliminar registros directamente desde la interfaz web del proyecto **Alke Wallet**.

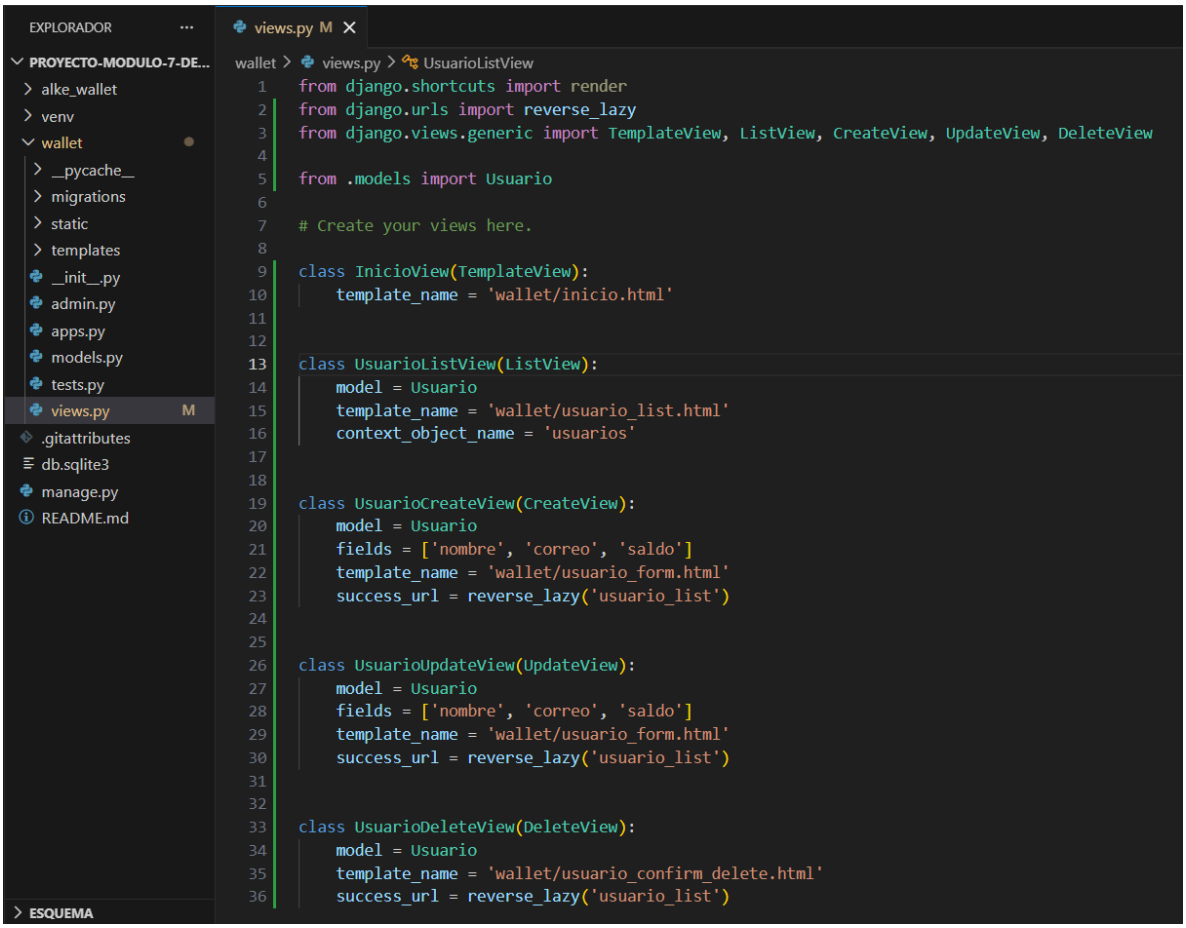
Para ello, se utilizaron vistas basadas en clases (Class-Based Views) provistas por Django, específicamente **ListView**, **CreateView**, **UpdateView** y **DeleteView**. Estas vistas permiten encapsular la lógica necesaria para cada operación CRUD, reduciendo código repetitivo y facilitando el mantenimiento del proyecto.

Además, se configuraron las rutas correspondientes en el archivo **urls.py** de la aplicación **wallet**, permitiendo acceder a cada operación mediante URLs específicas. Posteriormente, se crearon los templates necesarios para mostrar la lista de usuarios, el formulario de creación y edición, y la confirmación de eliminación.

Con esta implementación, el sistema ya permite gestionar usuarios desde una interfaz web propia, integrando el modelo **Usuario** con vistas, rutas y plantillas HTML.

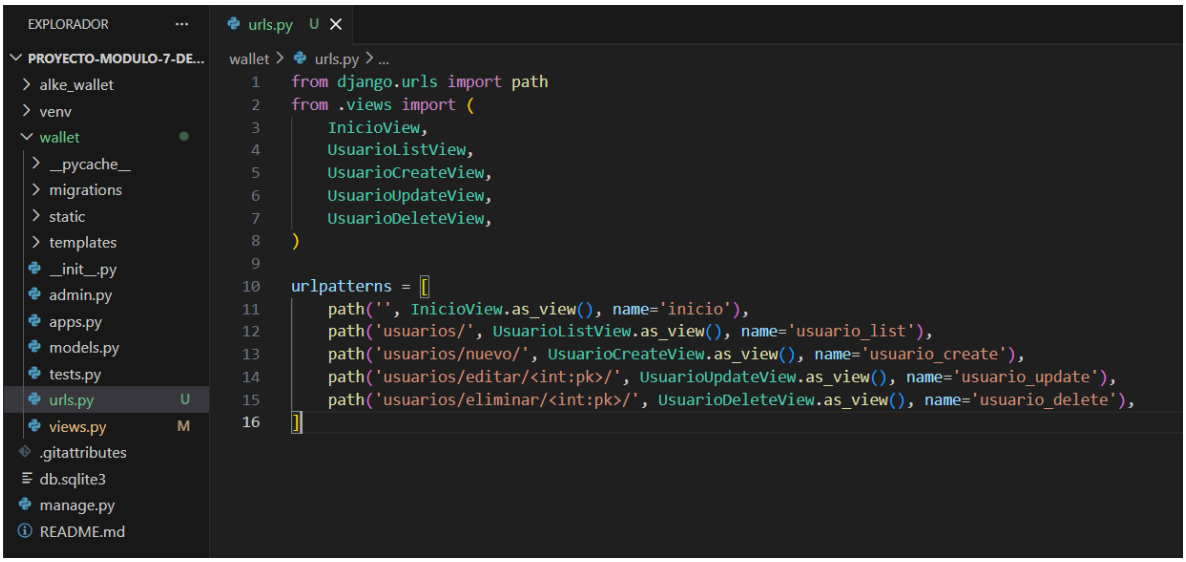
1. Creación de vistas CRUD para Usuario

Se implementaron las vistas basadas en clases necesarias para realizar las operaciones CRUD del modelo **Usuario**.



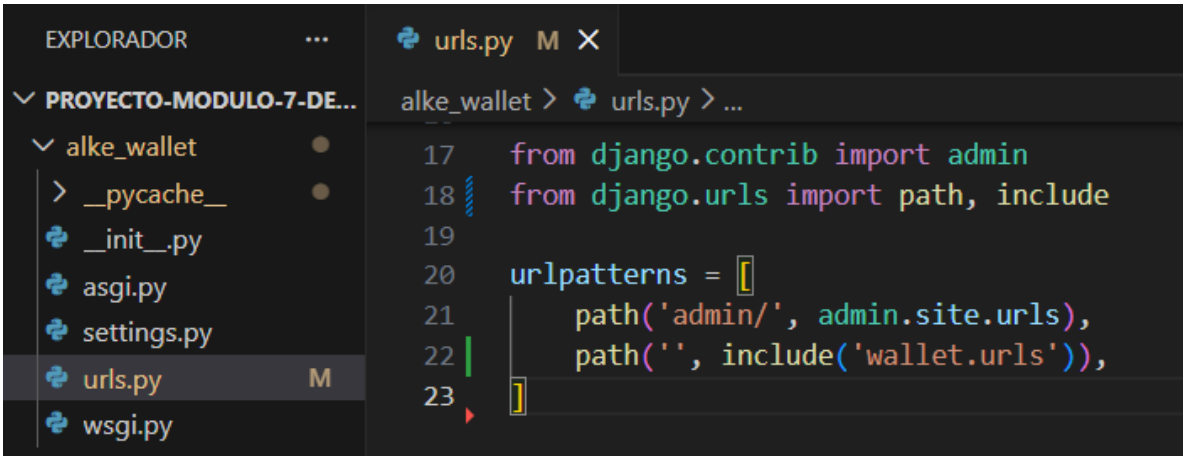
2. Configuración de rutas para Usuario

Se creó el archivo **urls.py** en la aplicación **wallet** para la configuración de las rutas necesarias, permitiendo acceder a las operaciones de listado, creación, edición y eliminación de usuarios.



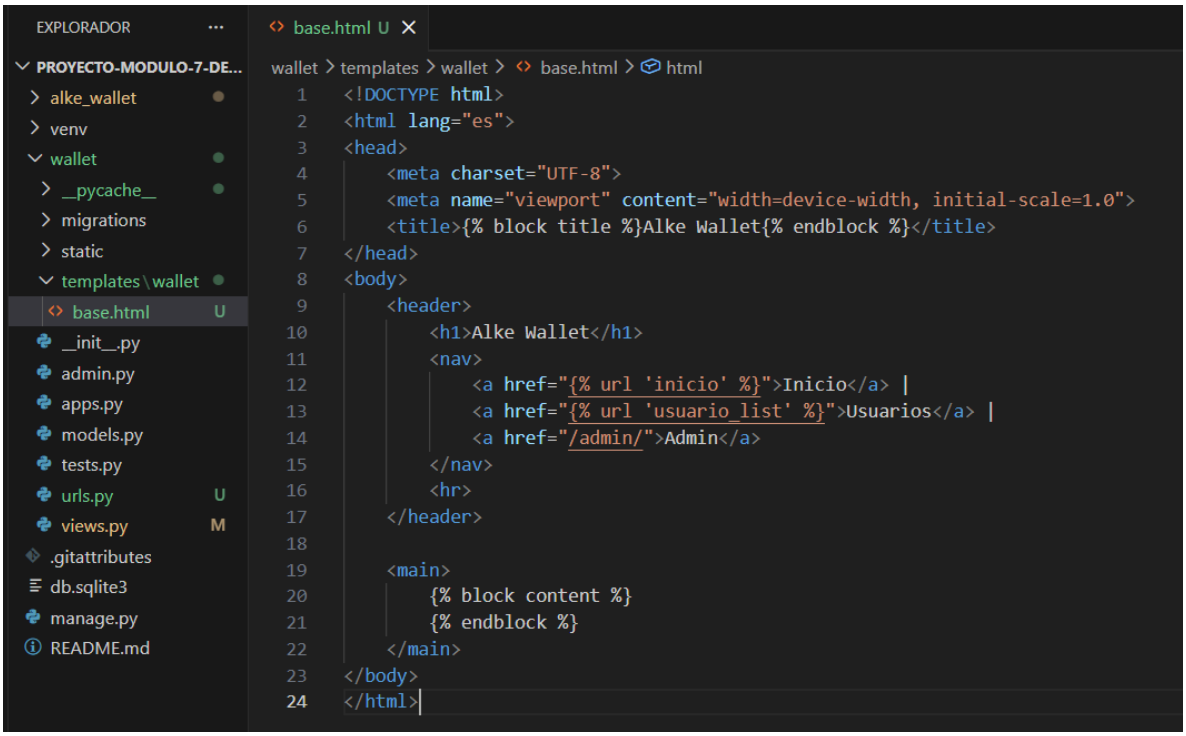
3. Conexión de URLs de la app con el proyecto

Se integraron las rutas de la app **wallet** dentro del archivo principal **alke_wallet/urls.py**, utilizando la función **include()**.



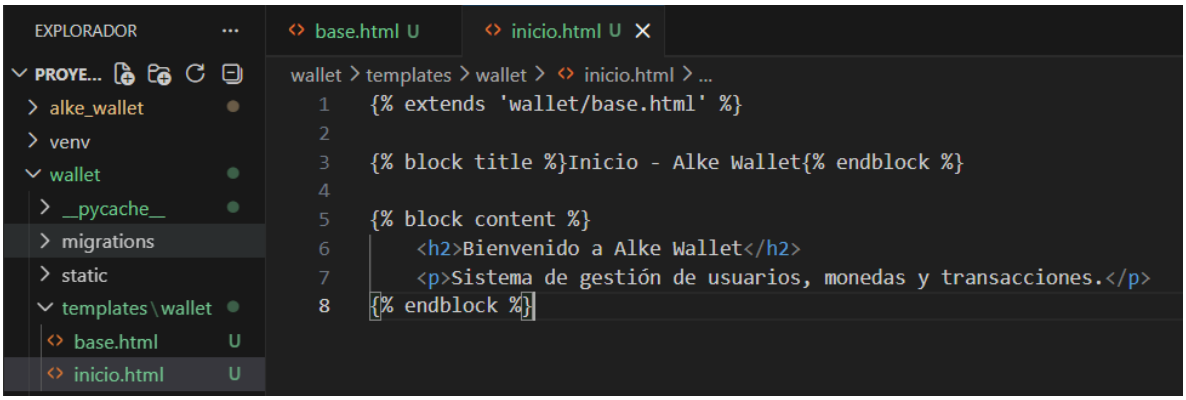
4. Creación del template base

Se creó una plantilla base reutilizable llamada **base.html**, la cual servirá como estructura común para las demás páginas del sistema.



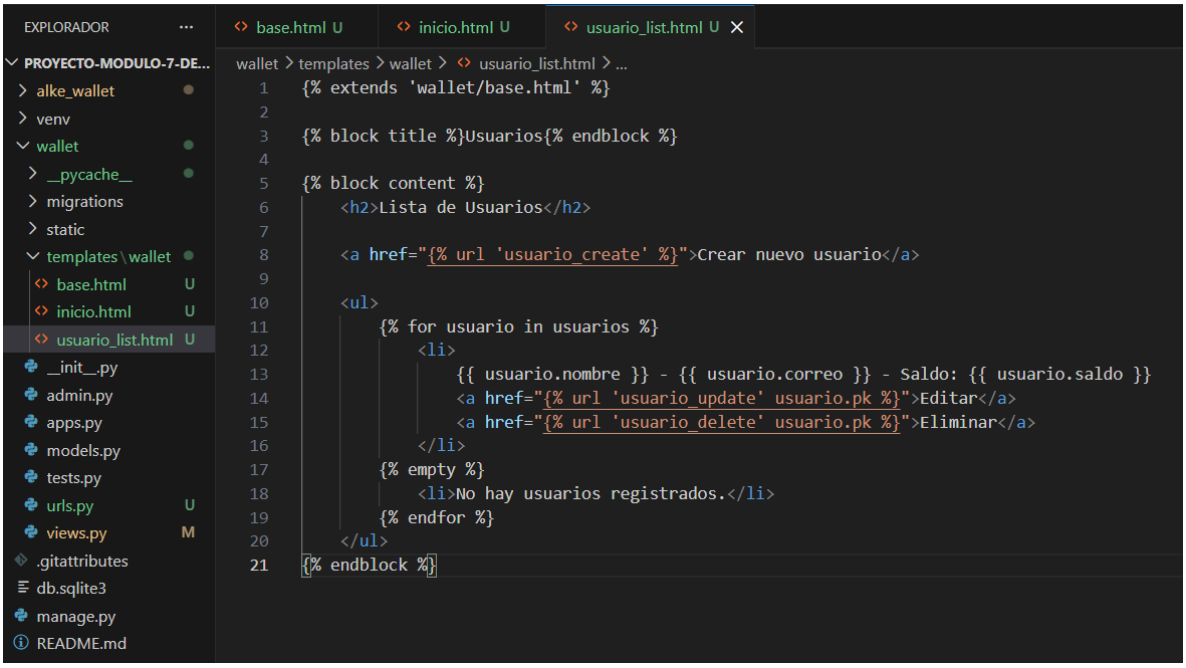
5. Creación de la vista de inicio

Se creó una página inicial para el sistema.



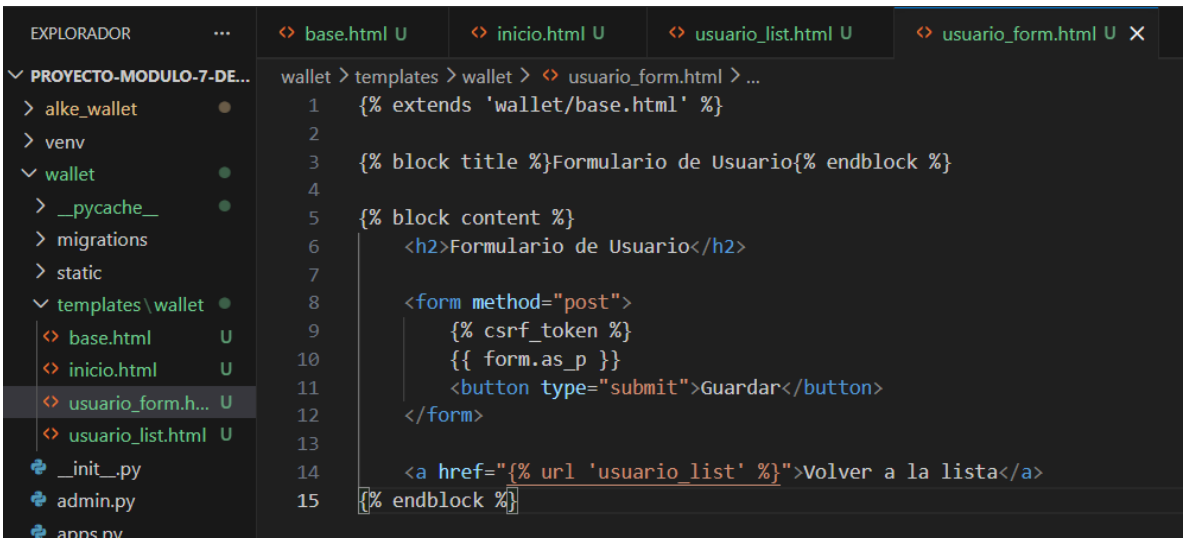
6. Template de listado de usuarios

Se creó el template encargado de mostrar todos los usuarios almacenados en la base de datos.



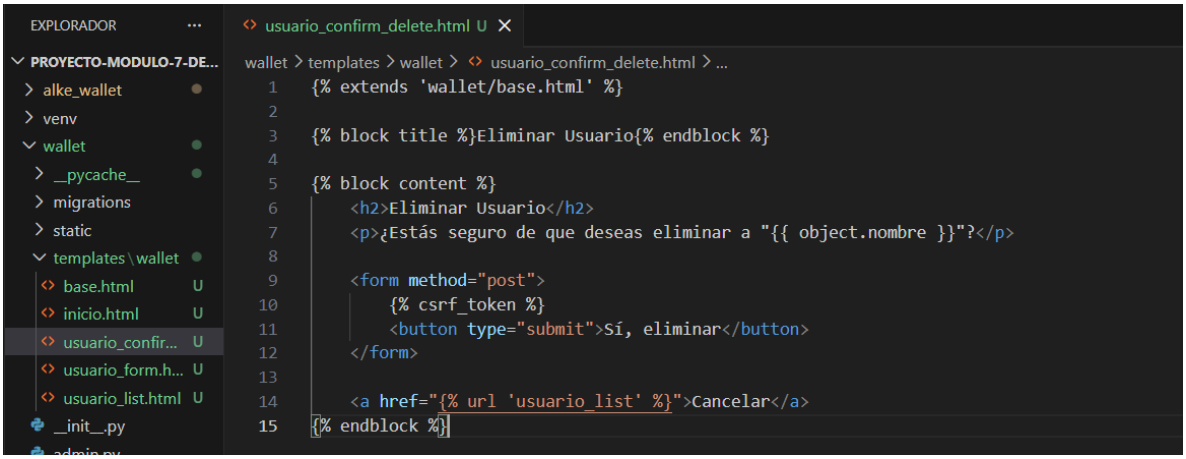
7. Template del formulario de usuario

Se creó una plantilla reutilizable para las operaciones de creación y edición de usuarios.



8. Template de confirmación de eliminación

Se creó una vista de confirmación para eliminar usuarios de forma segura.



9. Verificación del CRUD en el navegador

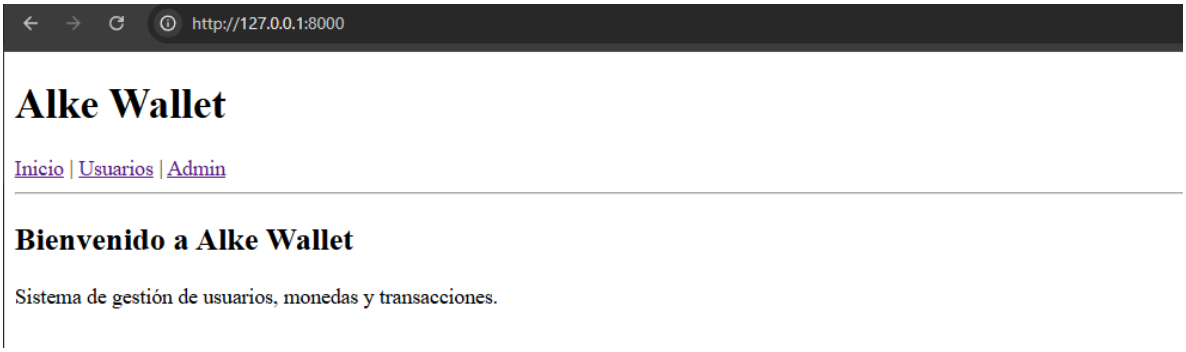
Una vez implementadas las vistas, rutas y templates, se ejecutó el servidor de desarrollo y se comprobó el funcionamiento del CRUD de usuarios desde la interfaz web.

Comandos utilizados:

- python manage.py runserver

Prueba de rutas:

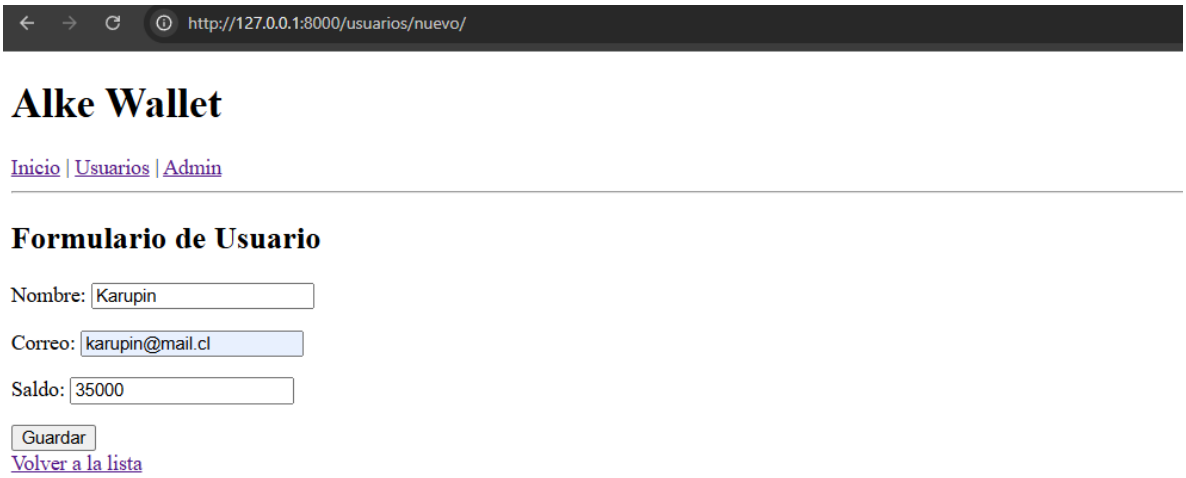
- http://127.0.0.1:8000/



- http://127.0.0.1:8000/usuarios/

- http://127.0.0.1:8000/usuarios/nuevo/

Prueba de creación de nuevo usuario



Al guardar redirige a lista de usuarios actualizada

http://127.0.0.1:8000/usuarios/

Alke Wallet

Inicio | Usuarios | Admin

Lista de Usuarios

Crear nuevo usuario

- Ignacio Trujillo - ignacio@email.com - Saldo: 150000.00 Editar Eliminar
- Macarena Trujillo - macarena@email.com - Saldo: 230000.00 Editar Eliminar
- Ilonka Trujillo - ilonka@email.com - Saldo: 50000.00 Editar Eliminar
- Karupin - karupin@mail.cl - Saldo: 35000.00 Editar Eliminar

Prueba de edición de datos de Ignacio Trujillo

http://127.0.0.1:8000/usuarios/editar/1/

Alke Wallet

Inicio | Usuarios | Admin

Formulario de Usuario

Nombre: Ignacio Trujillo Leiva

Correo: ignacioTrujilloLeiva@email.

Saldo: 500000,00

Guardar

Volver a la lista

Al guardar redirecciona a lista de usuario con las modificaciones indicadas

http://127.0.0.1:8000/usuarios/

Alke Wallet

Inicio | Usuarios | Admin

Lista de Usuarios

Crear nuevo usuario

- Ignacio Trujillo Leiva - ignacioTrujilloLeiva@email.cl - Saldo: 500000.00 Editar Eliminar
- Macarena Trujillo - macarena@email.com - Saldo: 230000.00 Editar Eliminar
- Ilonka Trujillo - ilonka@email.com - Saldo: 50000.00 Editar Eliminar
- Karupin - karupin@mail.cl - Saldo: 35000.00 Editar Eliminar

Prueba de eliminación de usuario Karupin

Al presionar se redirige a la ventana de confirmación y al confirmar con el botón “si, eliminar” redirige a lista de usuarios

5.7 Implementación del CRUD de transacciones

En esta etapa se implementó un sistema CRUD para el modelo **Transaccion**, el cual permitirá crear, visualizar, actualizar y eliminar registros desde la interfaz web del proyecto **Alke Wallet**.

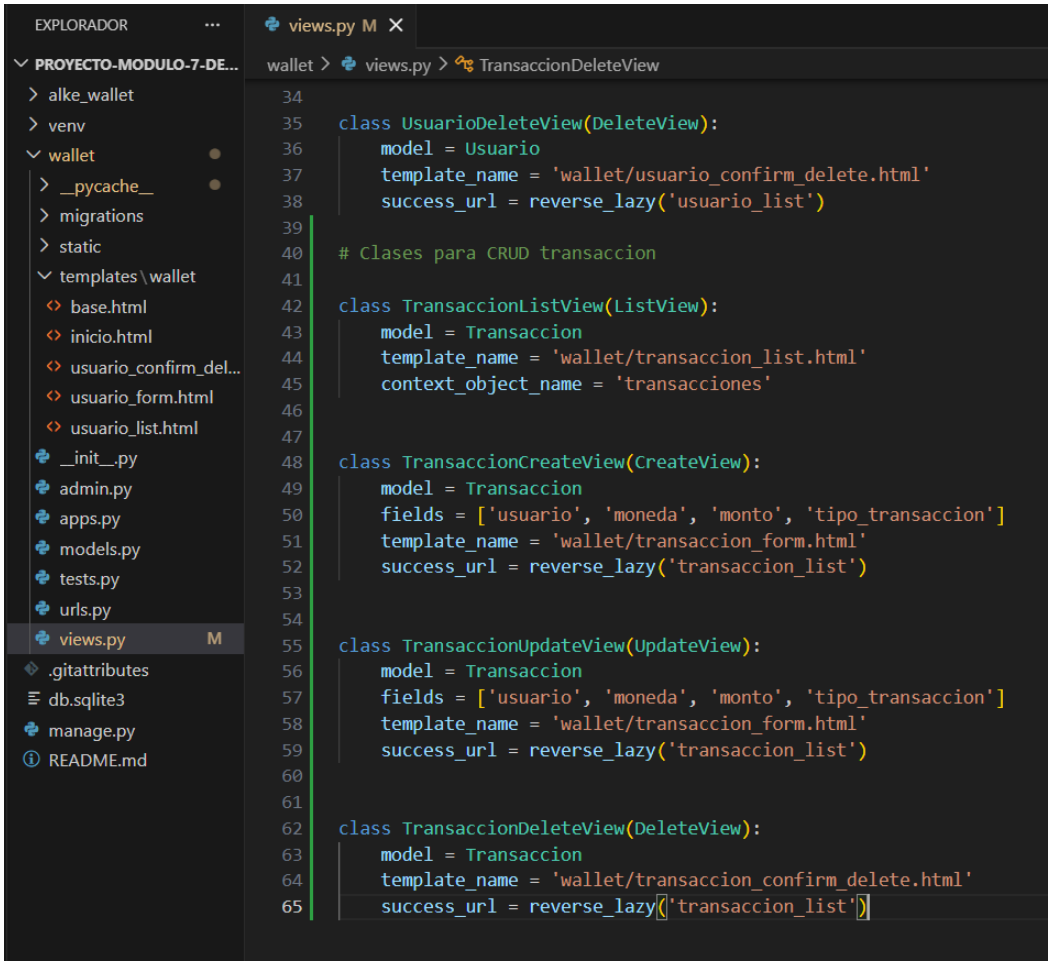
A diferencia del CRUD de usuarios, en este caso se trabajó con un modelo que posee relaciones con otras entidades del sistema, siendo estas **Usuario** y **Moneda**, mediante claves foráneas. Esto permitió evidenciar cómo Django maneja relaciones entre modelos y cómo estas se integran dentro de formularios y vistas mediante el ORM.

Para implementar esta funcionalidad se utilizaron vistas basadas en clases (**ListView**, **CreateView**, **UpdateView** y **DeleteView**), además de rutas específicas en el archivo **urls.py** de la aplicación **wallet**. Posteriormente, se desarrollaron los templates necesarios para mostrar el listado de transacciones, el formulario de creación y edición, y la confirmación de eliminación.

Con esta implementación, el sistema es capaz de gestionar operaciones financieras desde la interfaz web, utilizando formularios dinámicos conectados a modelos relacionados.

1. Incorporación de vistas CRUD para Transaccion

Se implementaron las vistas necesarias para gestionar registros del modelo **Transaccion** utilizando vistas basadas en clases.



2. Configuración de rutas para las transacciones

Se agregaron nuevas rutas en el archivo **wallet/urls.py** para permitir la navegación entre las distintas operaciones del CRUD de transacciones.

```python
from django.urls import path
from .views import (
    InicioView,
    UsuarioListView,
    UsuarioCreateView,
    UsuarioUpdateView,
    UsuarioDeleteView,
    TransaccionListView,
    TransaccionCreateView,
    TransaccionUpdateView,
    TransaccionDeleteView,
)

urlpatterns = [
    path('', InicioView.as_view(), name='inicio'),

    path('usuarios/', UsuarioListView.as_view(), name='usuario_list'),
    path('usuarios/nuevo/', UsuarioCreateView.as_view(), name='usuario_create'),
    path('usuarios/editar/<int:pk>/', UsuarioUpdateView.as_view(), name='usuario_update'),
    path('usuarios/eliminar/<int:pk>/', UsuarioDeleteView.as_view(), name='usuario_delete'),

    path('transacciones/', TransaccionListView.as_view(), name='transaccion_list'),
    path('transacciones/nueva/', TransaccionCreateView.as_view(), name='transaccion_create'),
    path('transacciones/editar/<int:pk>/', TransaccionUpdateView.as_view(), name='transaccion_update'),
    path('transacciones/eliminar/<int:pk>/', TransaccionDeleteView.as_view(), name='transaccion_delete'),
]
```

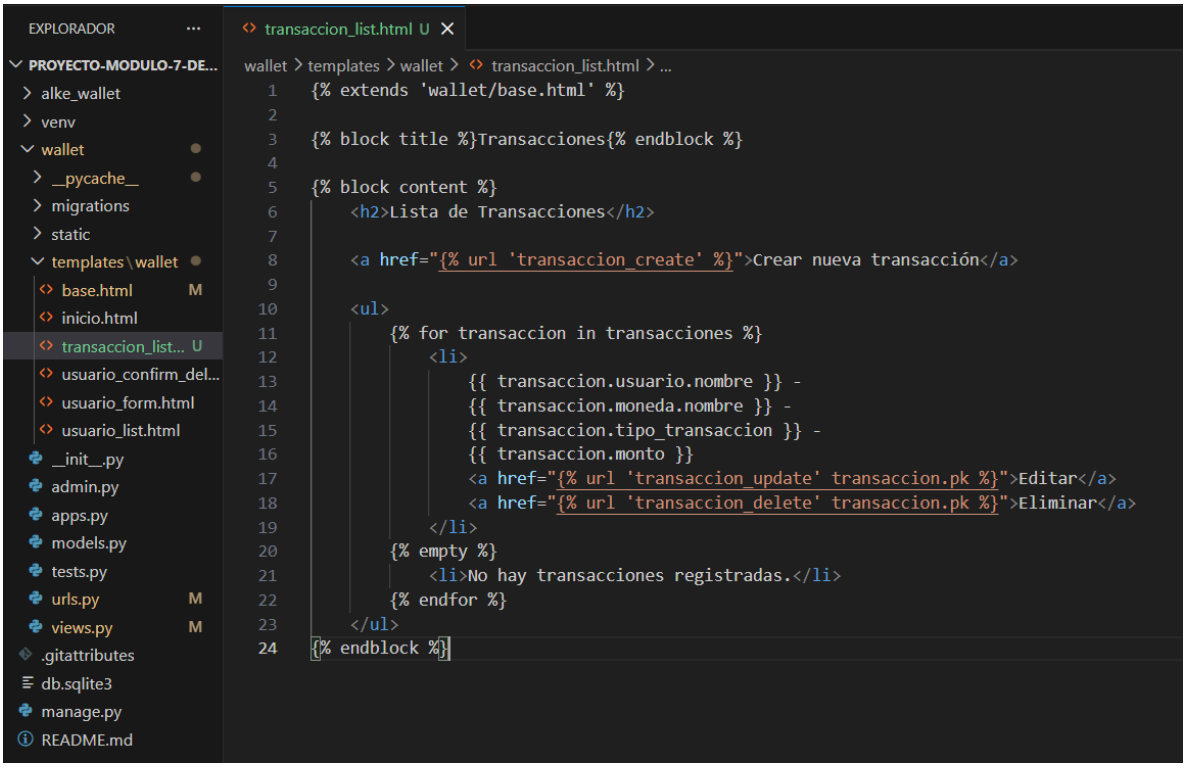
3. Actualización del menú de navegación

Se actualizó la plantilla base del sistema para incorporar el acceso al módulo de transacciones desde el menú principal.

```html
<!DOCTYPE html>
<html lang="es">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Alke Wallet{% endblock %}</title>
</head>

<body>
    <header>
        <h1>Alke Wallet</h1>
        <nav>
            <a href="{% url 'inicio' %}">Inicio</a> |
            <a href="{% url 'usuario_list' %}">Usuarios</a> |
            <a href="{% url 'transaccion_list' %}">Transacciones</a> |
            <a href="/admin/">Admin</a>
        </nav>
        <hr>
    </header>

    <main>
        {% block content %}
        {% endblock %}
    </main>
</body>

</html>
```

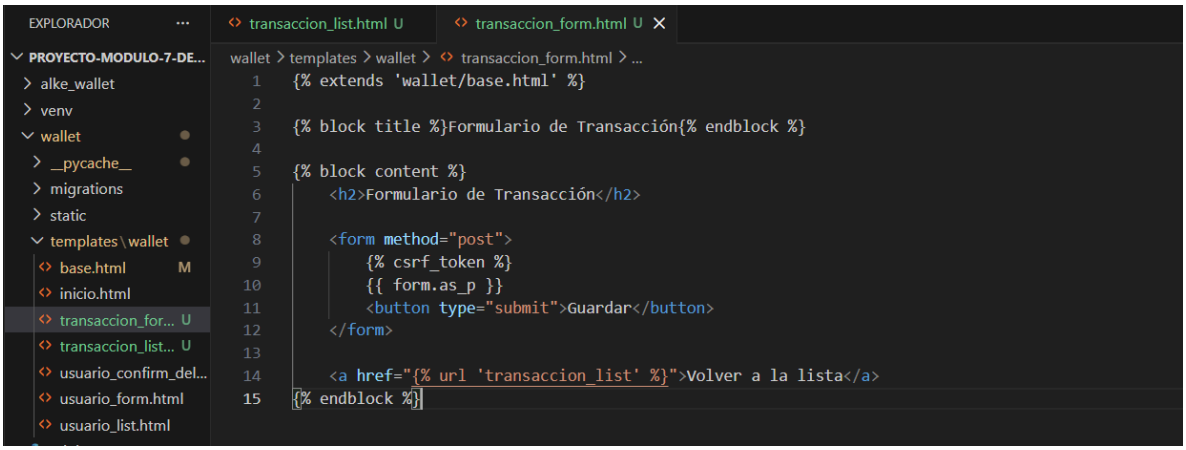
4. Creación del template de listado de transacciones

Se creó una plantilla para mostrar todas las transacciones registradas en la base de datos.



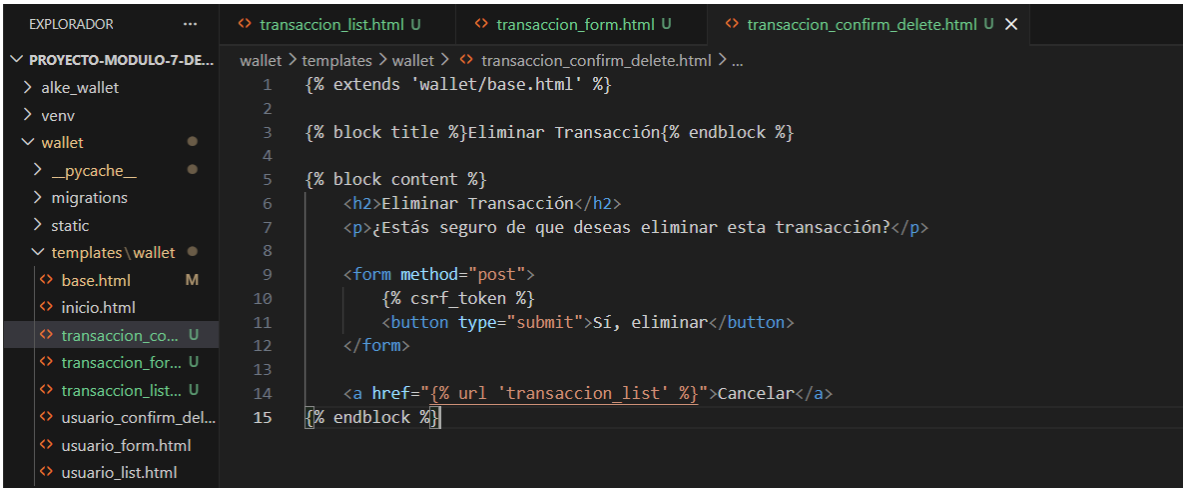
5. Creación del formulario de transacción

Se creó un template para las operaciones de creación y edición de transacciones.



6. Confirmación de eliminación de transacción

Se creó una vista de confirmación para la eliminación segura de registros del modelo Transaccion.



7. Verificación del CRUD de transacciones

Una vez configuradas las vistas, rutas y templates, se ejecutó nuevamente el servidor de desarrollo para comprobar el correcto funcionamiento del CRUD de transacciones desde el navegador.

Comandos utilizados:

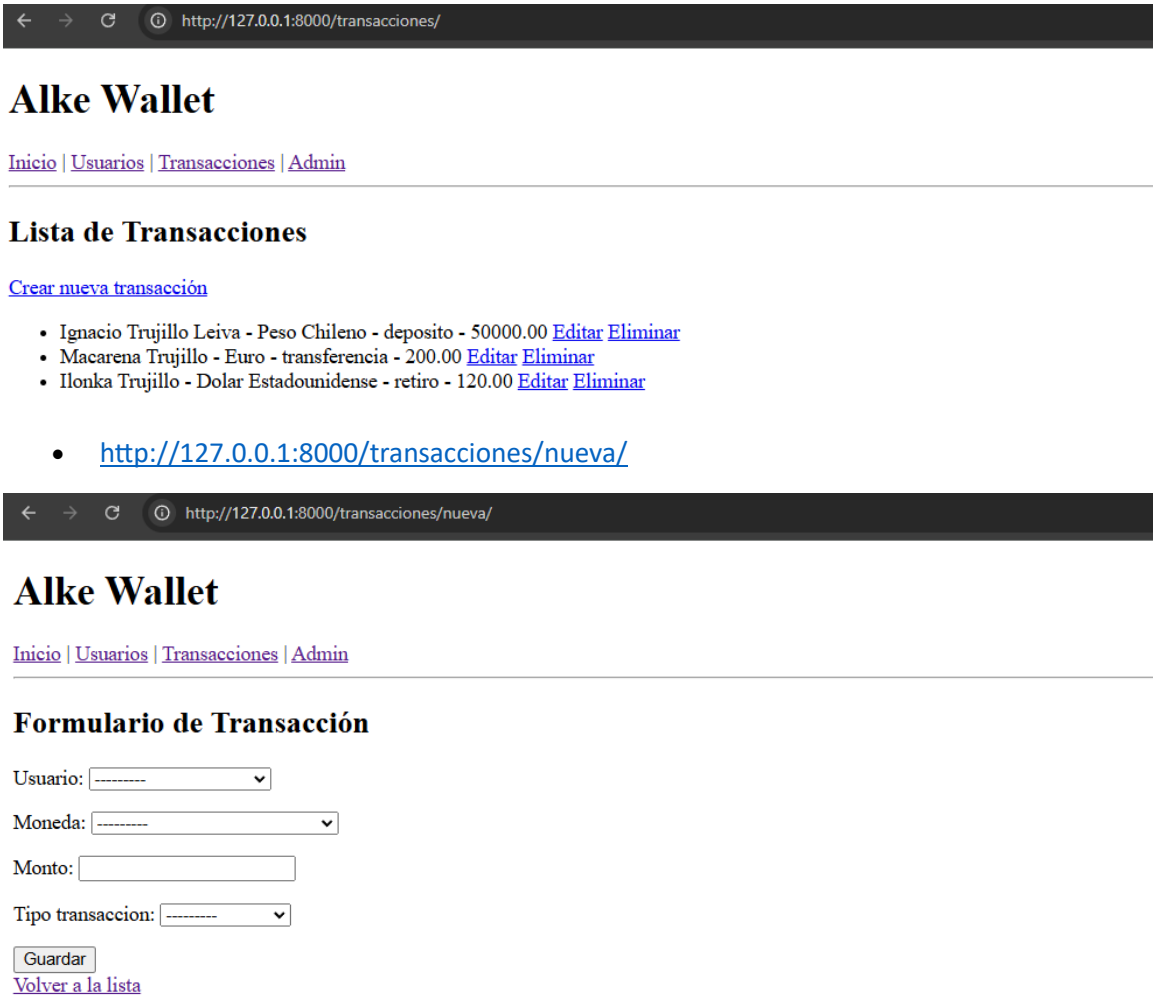
- python manage.py runserver

Prueba de rutas:

- http://127.0.0.1:8000/

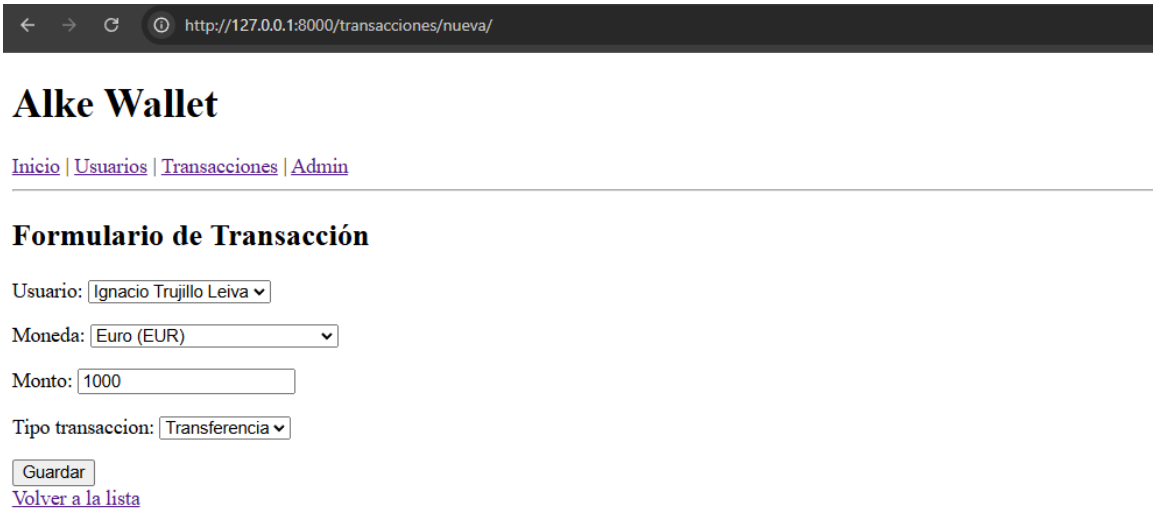
Rutas de prueba:

- http://127.0.0.1:8000/transacciones/

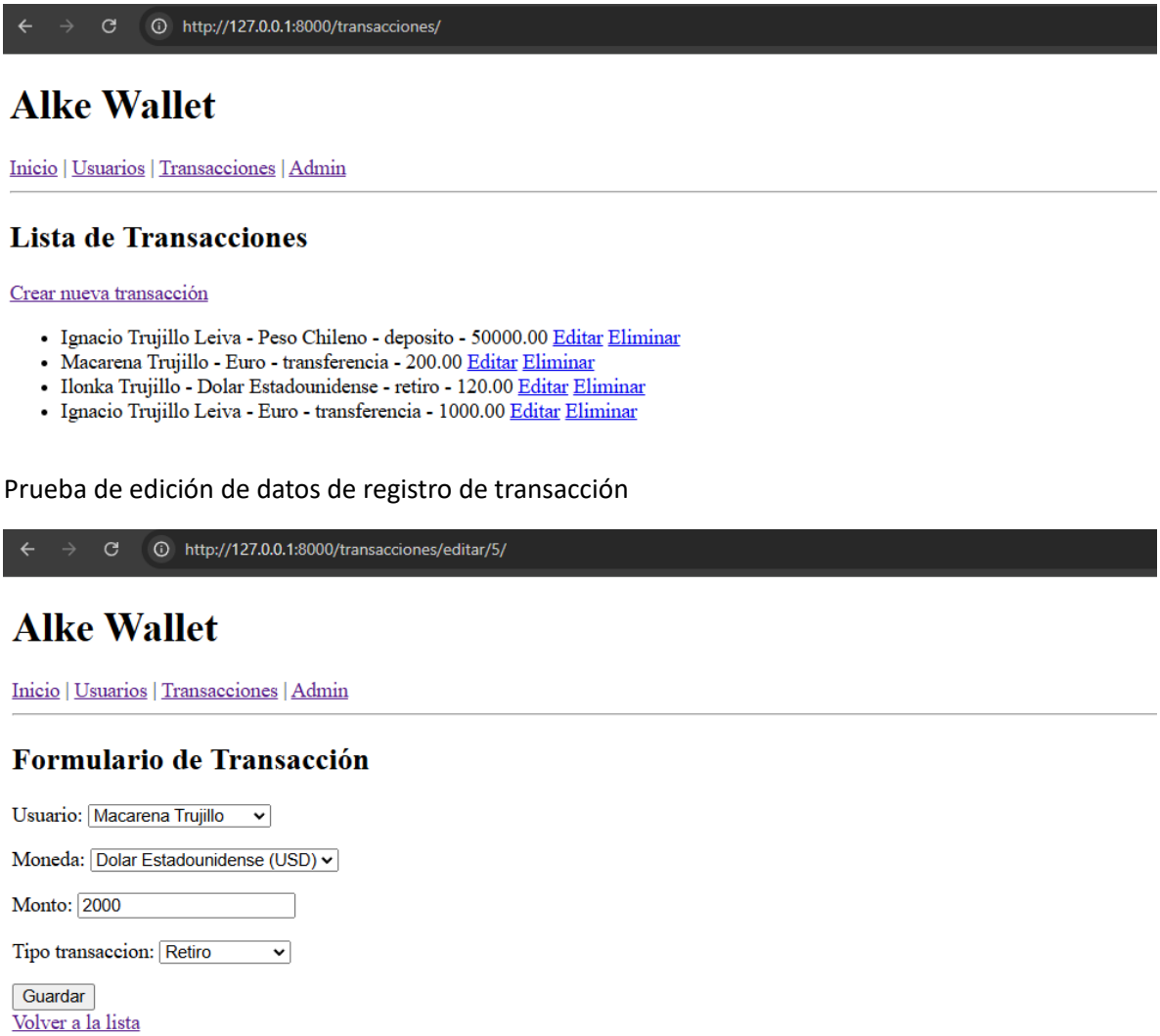


- http://127.0.0.1:8000/transacciones/nueva/

Prueba de creación de nueva transacción



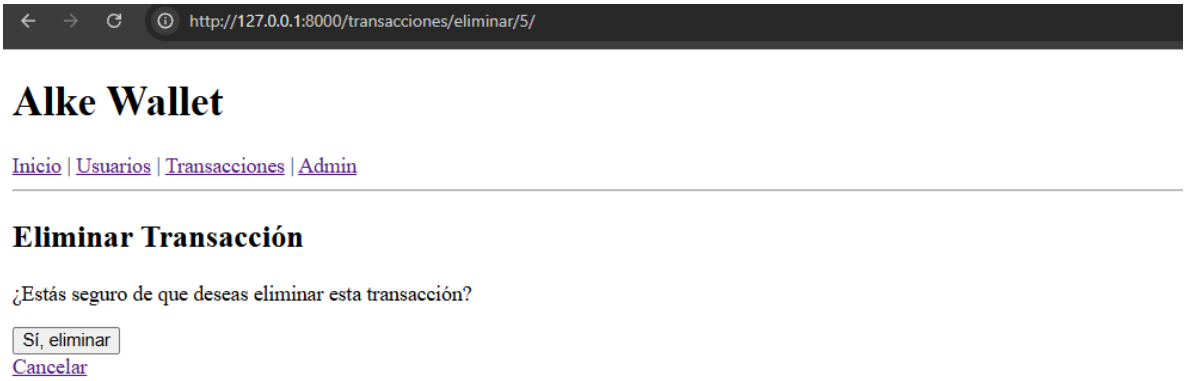
Una vez guardado el registro, se redirecciona a la lista de transacciones con el registro actualizado



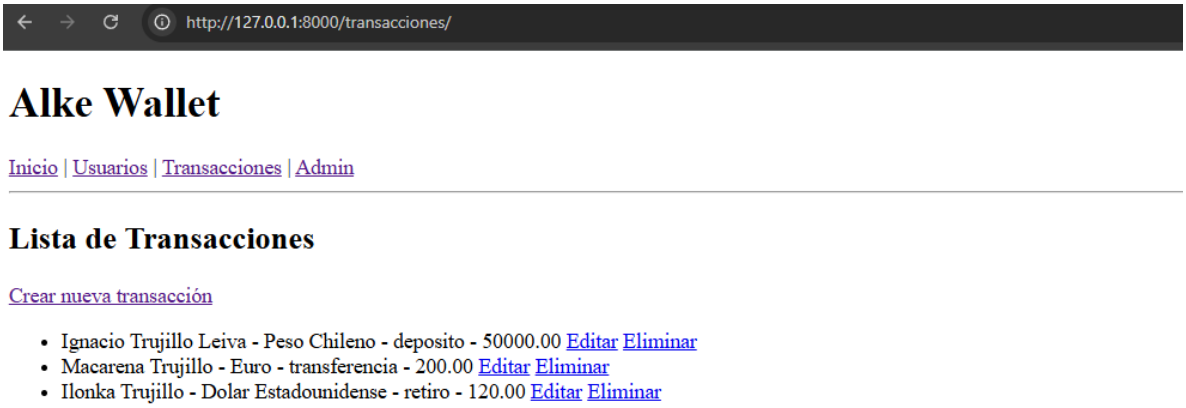
Prueba de edición de datos de registro de transacción

Al guardar se redirecciona a listado de registro de transacciones con registro actualizado, cabe destacar que a pesar de no tener sentido la modificación de registros de transacciones en una aplicación Fintech, esta opción se dejó por el concepto de poseer las operaciones CRUD

Prueba de eliminación de registro



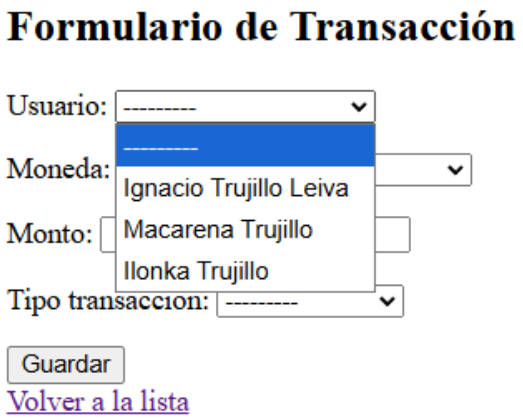
Al confirmar la eliminación del registro, se redirecciona a la lista de transacciones

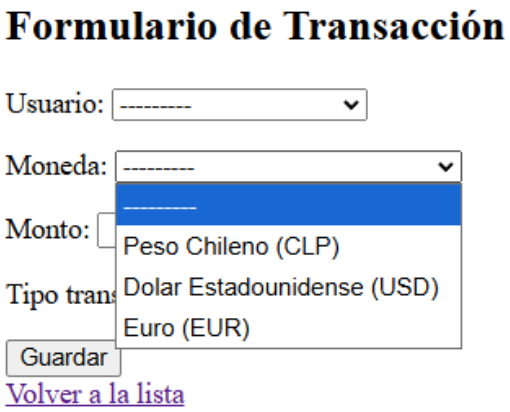


8. Validación de relaciones en los formularios

En el formulario de transacción se comprobó que Django mostrara automáticamente listas desplegables para seleccionar el **Usuario** y la **Moneda**, gracias a las relaciones definidas mediante **ForeignKey**.

Esto permite verificar que las relaciones entre modelos se encontraban correctamente integradas al flujo del sistema y que el ORM gestionaba adecuadamente la asociación entre entidades.





5.8 Implementación de estilos visuales

En esta etapa se incorporaron mejoras visuales al proyecto con el objetivo de ofrecer una interfaz más moderna, intuitiva y atractiva para el usuario.

Se utilizó el framework Bootstrap para facilitar la estructura responsiva del sistema, complementado con una hoja de estilos personalizada (style.css) que permitió definir una identidad visual propia.

Entre las principales mejoras implementadas se destacan:

- Incorporación de una imagen de fondo con diseño moderno tipo aplicación financiera.
- Creación de contenedores tipo “card” centrados para mejorar la experiencia del usuario.
- Diseño de botones con gradientes de color, generando una interfaz más dinámica.
- Estilización de formularios con bordes redondeados y mejor distribución.
- Presentación de listados de transacciones de forma ordenada y clara.

5.9 Integración final, navegación y verificación general del sistema

En esta etapa se realizó la integración final de los componentes principales del proyecto **Alke Wallet**, con el objetivo de comprobar el funcionamiento general del sistema y validar la coherencia entre la base de datos, las vistas, las rutas y los templates desarrollados.

Una vez implementados los modelos, aplicadas las migraciones, configurado el panel administrativo y desarrollados los módulos CRUD para **Usuario** y **Transaccion** e implementación de estilos visuales, se procedió a verificar el flujo completo de navegación dentro de la aplicación. Para ello, se utilizó una plantilla base común (**base.html**), la cual permitió centralizar el menú de navegación y facilitar el acceso a las distintas secciones del sistema.

Durante esta etapa se comprobó que el usuario pudiera navegar desde la página de inicio hacia los listados de usuarios y transacciones, así como acceder a los formularios de creación, edición y eliminación de registros de manera correcta. También se validó que las operaciones realizadas desde la interfaz web se reflejaran adecuadamente en la base de datos.

Además, se verificó que las relaciones entre modelos funcionaran correctamente en los formularios y listados, especialmente en el caso de las transacciones, donde fue posible asociar un usuario y una moneda mediante listas desplegables generadas automáticamente por Django.

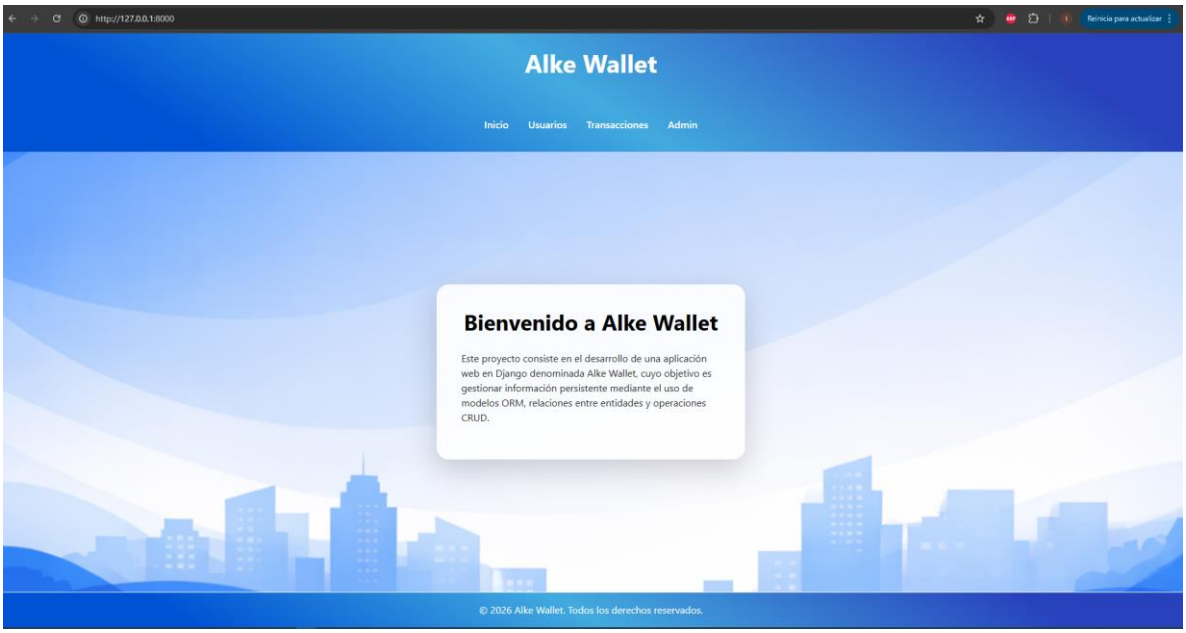
Finalmente, esta etapa permitió consolidar el flujo general del proyecto, demostrando que la aplicación cumple con los requerimientos del módulo al integrar de forma funcional la base de datos, el ORM, las relaciones simples y las operaciones CRUD solicitadas.

1. Verificación de la navegación principal

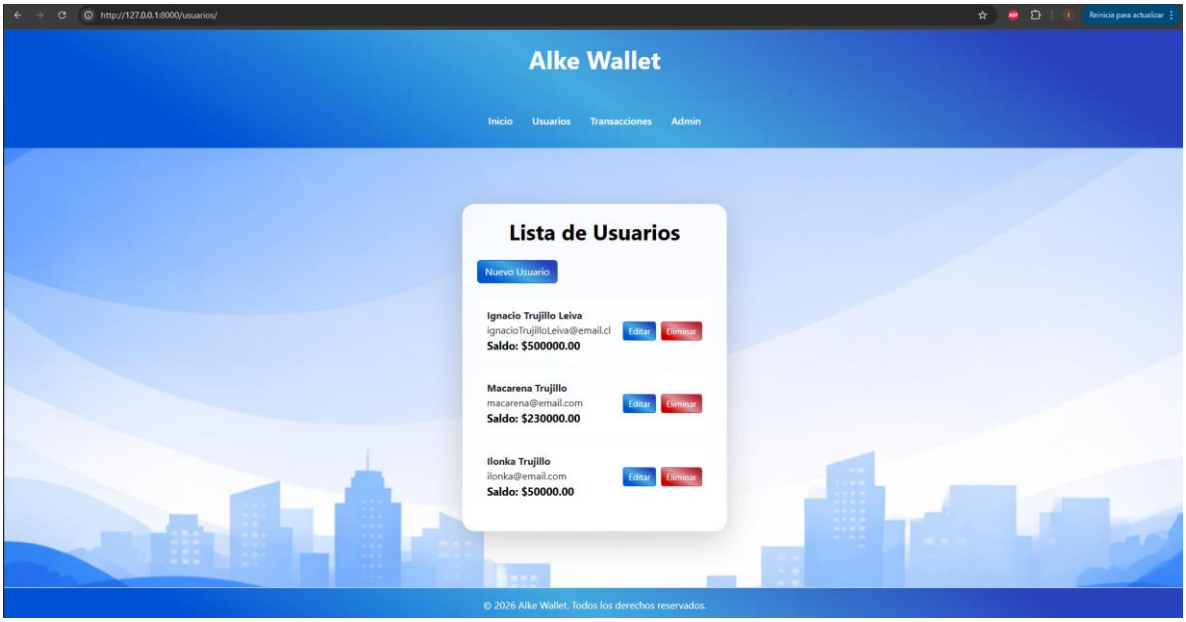
Se comprobó que el sistema permitiera navegar correctamente entre las distintas vistas implementadas a través del menú principal definido en base.html.

Se verificaron los accesos a:

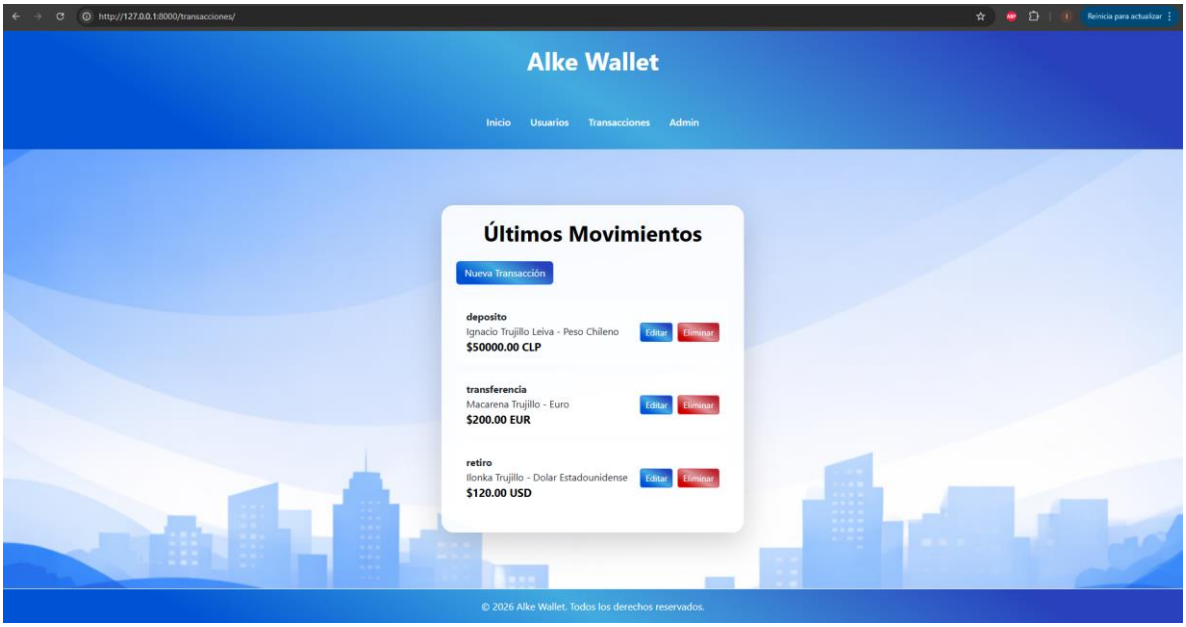
- Página de inicio



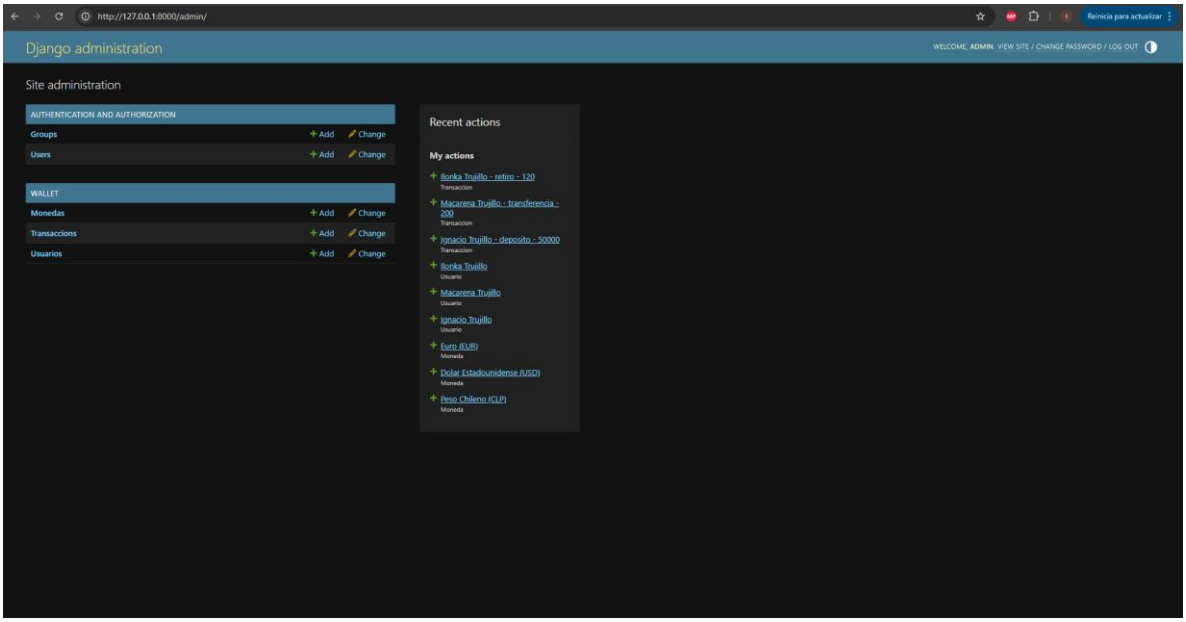
- Listado de usuarios



- Listado de transacciones



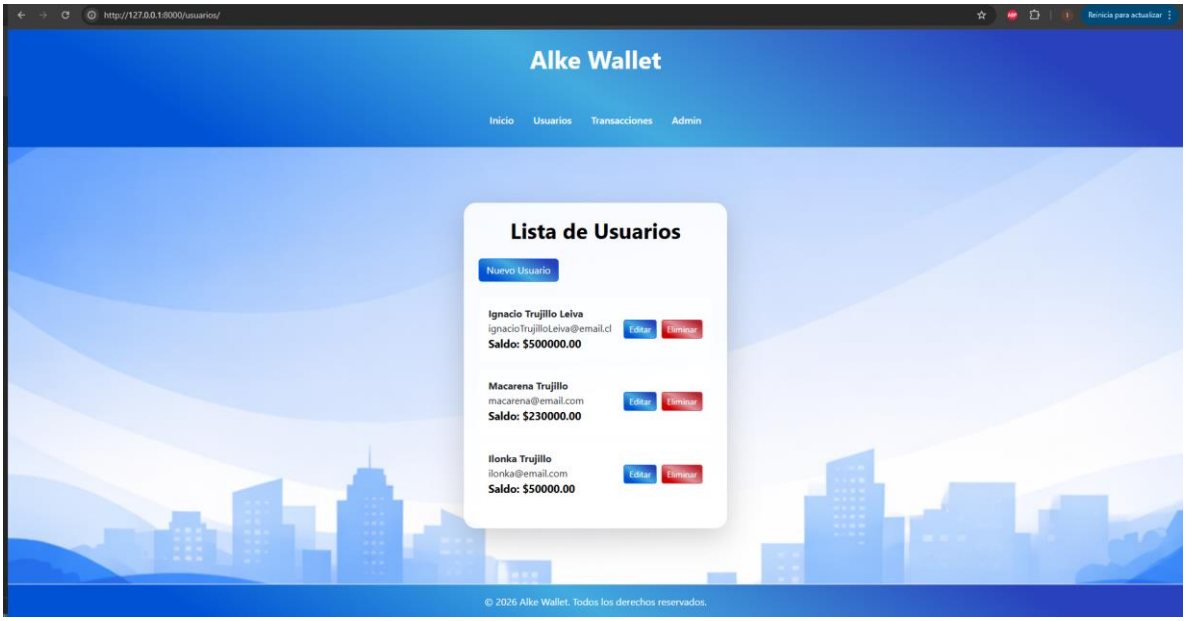
- Panel administrativo



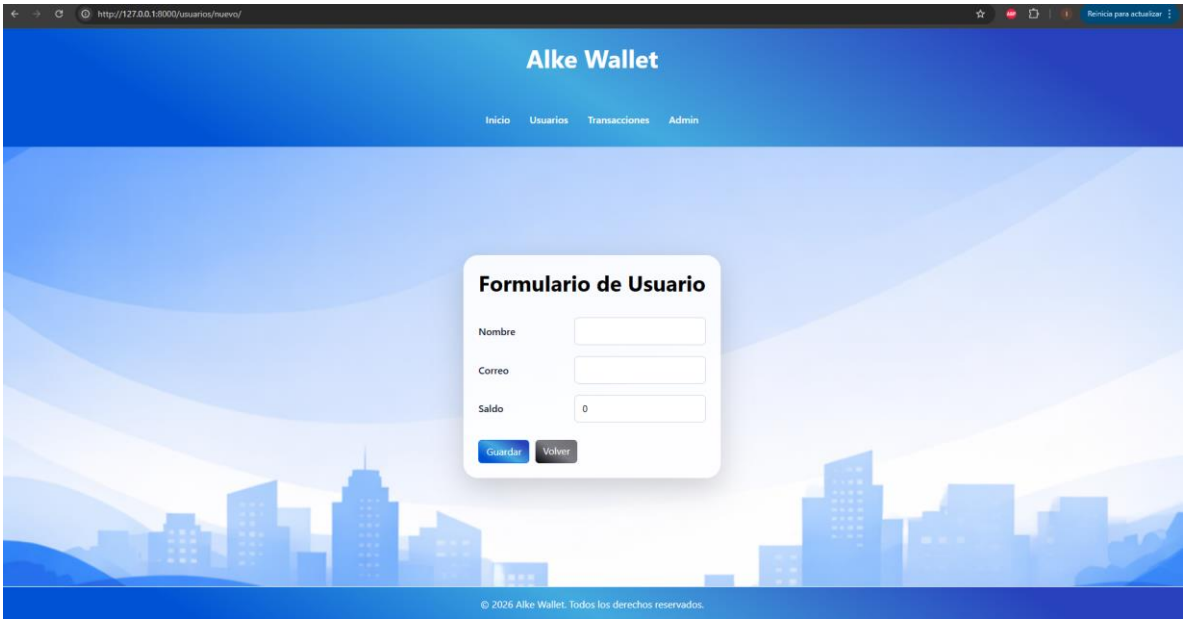
2. Validación del flujo CRUD de usuarios

Se revisó el comportamiento completo del CRUD del modelo Usuario, verificando que fuera posible:

- listar usuarios

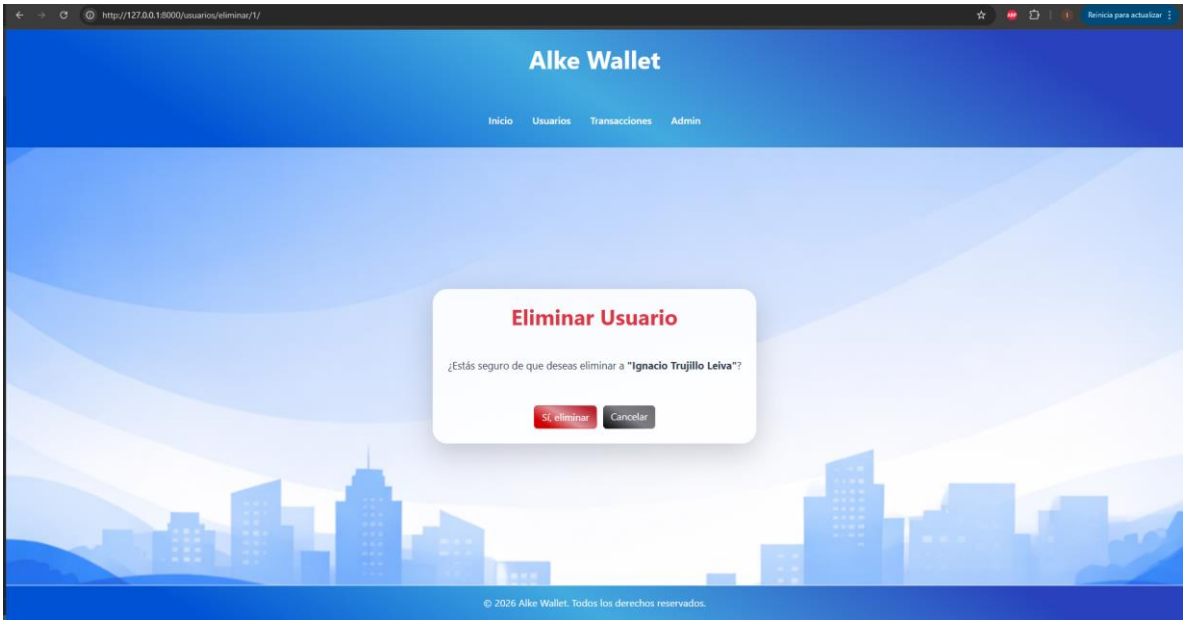


- crear nuevos registros



- editar registros existentes

- eliminar registros

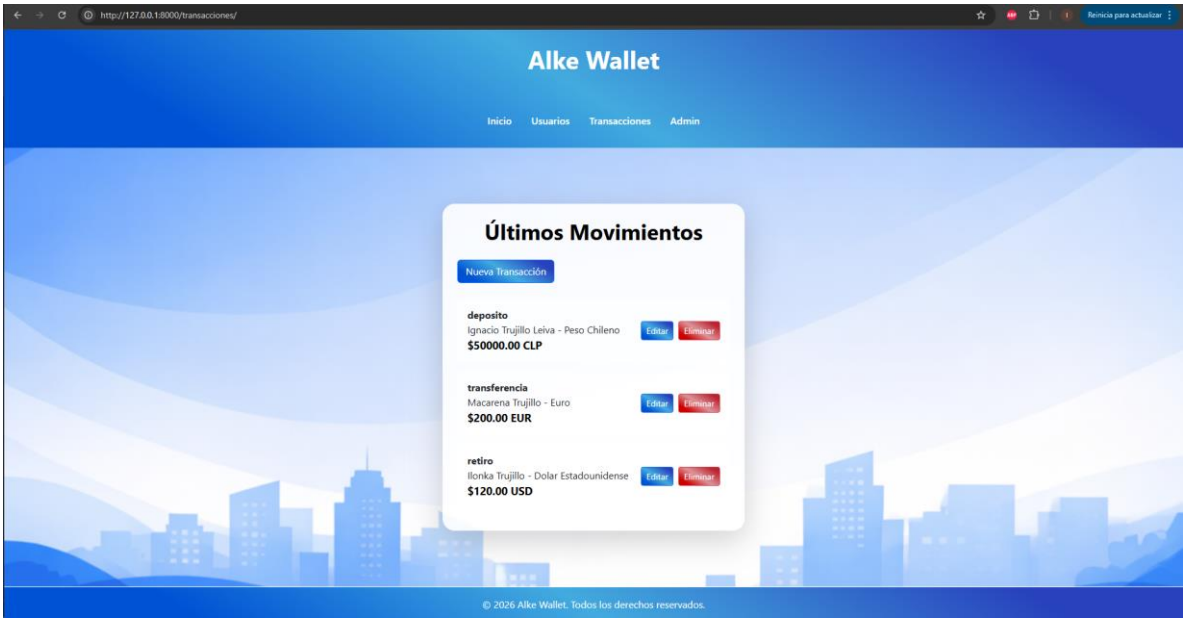


Además, se comprobó que luego de cada operación el sistema redirigiera correctamente al listado de usuarios.

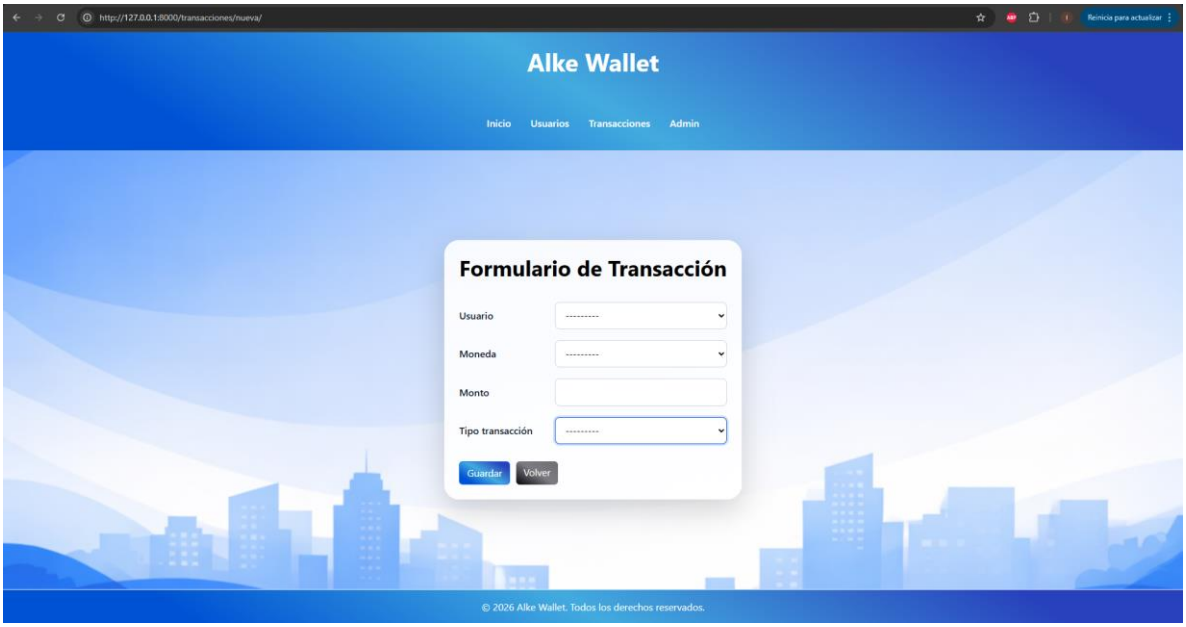
3. Validación del flujo CRUD de transacciones

Se revisó el funcionamiento completo del CRUD del modelo Transaccion, verificando que fuera posible:

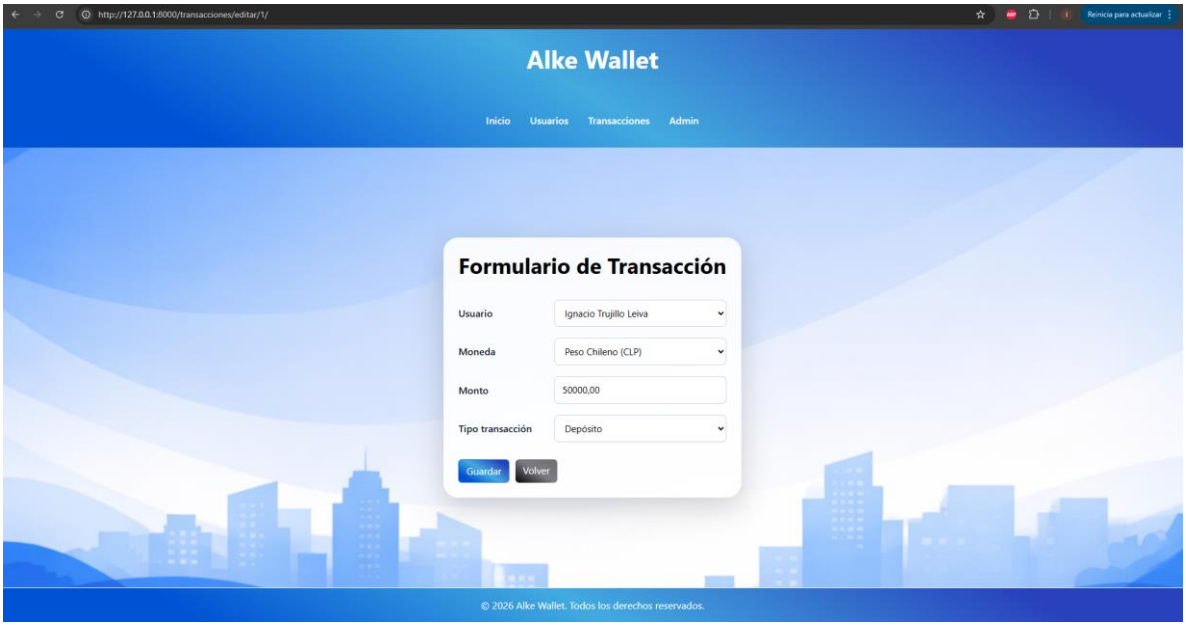
- listar transacciones



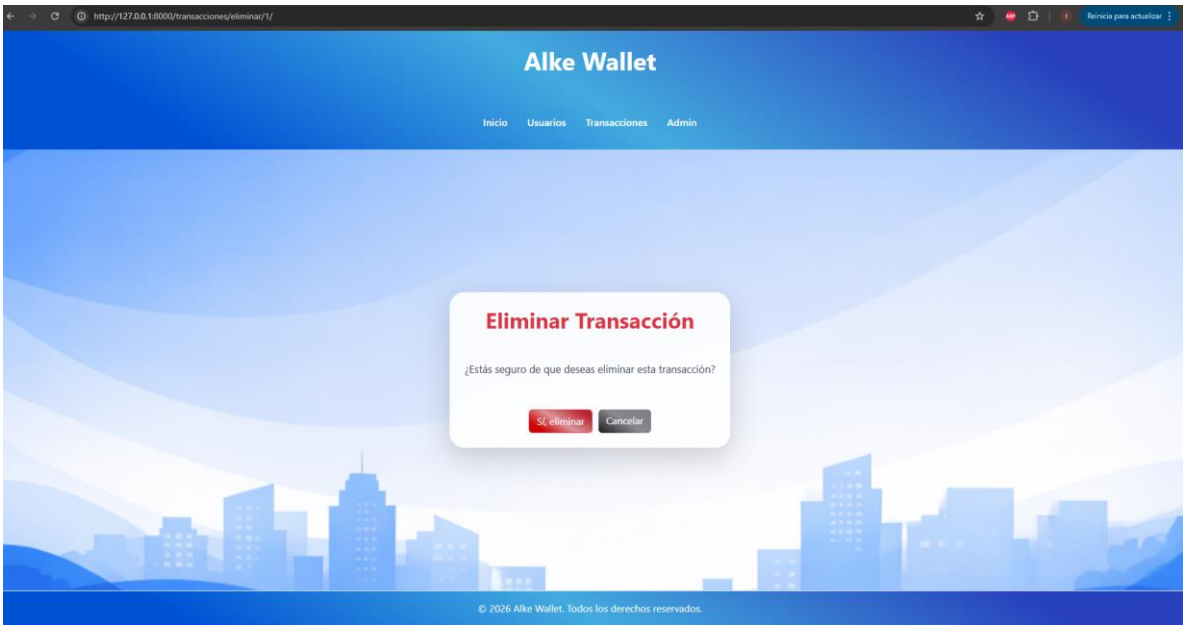
- crear nuevas transacciones



- editar registros existentes



- eliminar registros



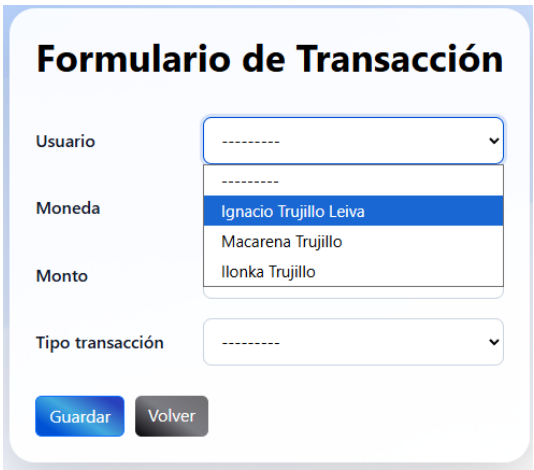
También se comprobó que el formulario mostrara correctamente las relaciones con Usuario y Moneda, mediante listas desplegables.

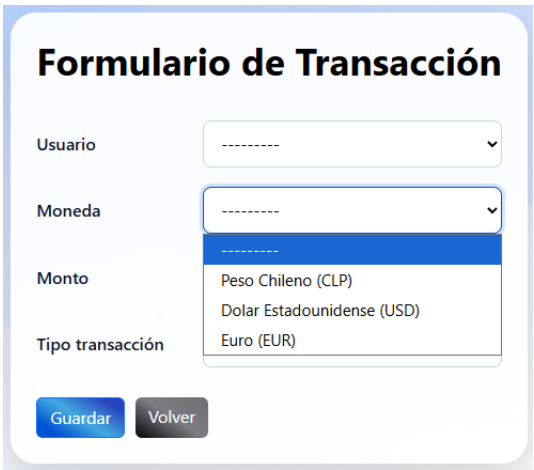
4. Verificación de relaciones entre modelos

Se validó que las relaciones definidas entre Usuario, Moneda y Transaccion funcionaran correctamente en la práctica. Esto se observó especialmente en el formulario de transacción, donde se pudo seleccionar un usuario y una moneda previamente registrados, comprobando así la integración entre las entidades.

De esta manera, se confirmó que:

- un usuario puede registrar múltiples transacciones
- una moneda puede estar asociada a múltiples transacciones





5. Verificación de persistencia de datos

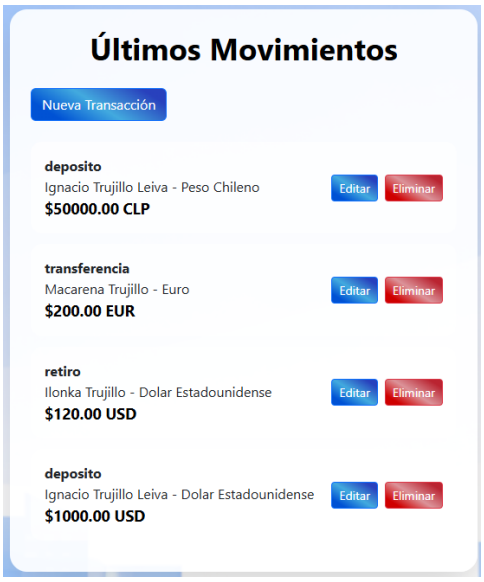
Se comprobó que los registros creados desde la interfaz web quedaran almacenados correctamente en la base de datos SQLite. Esta validación se realizó comparando la información visible en las vistas del sistema con los datos cargados previamente desde el panel administrativo y mediante formularios web.

Esto permitió confirmar que el sistema no solo mostraba información en pantalla, sino que también almacenaba y actualizaba los datos de forma persistente.

Action: --------- Run 0 of 4 selected

USUARIO	MONEDA	MONTO	TIPO TRANSACCION	FECHA
Ignacio Trujillo Leiva	Dolar Estadounidense (USD)	1000.00	Depósito	March 31, 2026, 5:04 a.m.
Ilonka Trujillo	Dolar Estadounidense (USD)	120.00	Retiro	March 30, 2026, 7:27 p.m.
Macarena Trujillo	Euro (EUR)	200.00	Transferencia	March 30, 2026, 7:26 p.m.
Ignacio Trujillo Leiva	Peso Chileno (CLP)	50000.00	Depósito	March 30, 2026, 7:25 p.m.

4 transaccions



6. Validación de integración entre modelo, vista y template

Se comprobó que cada operación realizada por el usuario siguiera correctamente el flujo de funcionamiento propio de Django:

1. El usuario accede a una URL
2. Django ejecuta la vista correspondiente
3. La vista consulta o modifica el modelo
4. El resultado se renderiza en un template HTML

Este flujo fue validado tanto para usuarios como para transacciones, comprobando que la arquitectura del sistema funciona de manera coherente y estable.

7. Conclusión

El desarrollo del proyecto **Alke Wallet** permitió aplicar de forma práctica los principales contenidos del módulo 7, integrando Django con una base de datos relacional mediante el uso del ORM. A lo largo del proceso se configuró el entorno de desarrollo, se creó la estructura del proyecto y de la aplicación principal, y se definieron modelos representativos capaces de almacenar información persistente.

Además, se implementaron relaciones simples entre entidades, permitiendo asociar usuarios, monedas y transacciones de forma coherente con la lógica del sistema. Estas relaciones fueron integradas correctamente mediante claves foráneas, lo que facilitó tanto la administración de datos como la construcción de formularios y vistas conectadas a la base de datos.

Por otra parte, se aplicaron migraciones para sincronizar el modelo lógico del proyecto con la estructura real de la base de datos, asegurando consistencia en el almacenamiento de la información. Posteriormente, se desarrollaron operaciones CRUD completas para los modelos principales, utilizando vistas basadas en clases, rutas específicas y templates HTML, lo que permitió gestionar registros directamente desde la interfaz web.

Finalmente, el proyecto también incorporó el uso del panel administrativo de Django para la carga y gestión de datos, lo que complementó la validación del sistema y facilitó las pruebas funcionales. En conclusión, este proyecto permitió consolidar los aprendizajes del módulo al desarrollar una aplicación web funcional, estructurada y conectada a base de datos, demostrando el uso integrado de modelos, relaciones, migraciones, CRUD y administración dentro del framework Django.